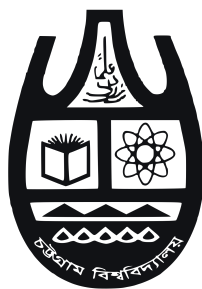


AdaPruner-KGQA: An Adaptive Search-Space Pruning Framework for Efficient Multi-Hop Knowledge Graph Question Answering



Md Sadman Sami Khan

Student ID: 21701081

Session: 2020-2021

Supervisor: Dr. Rudra Pratap Deb Nath

Department of Computer Science and Engineering
UNIVERSITY OF CHITTAGONG

This dissertation is submitted for the degree of
Bachelor of Science in Engineering (B.Sc. Engg.) in
Computer Science and Engineering

Report Code:

University of Chittagong

Department of Computer Science and Engineering

8th Semester B.Sc. Engineering Examination 2024

Course No.: CSE 800

Title: AdaPruner-KGQA: An Adaptive Search-Space Pruning Framework for Efficient Multi-Hop Knowledge Graph Question Answering

Report Code:

University of Chittagong

Department of Computer Science and Engineering

8th Semester B.Sc. Engineering Examination 2024

Course No.: CSE 800

Title: AdaPruner-KGQA: An Adaptive Search-Space Pruning Framework for Efficient Multi-Hop Knowledge Graph Question Answering

Student Name: Md Sadman Sami Khan

Student ID: 21701081

Session: 2020-2021

Hall: Shaheed Abdur Rab

Signature of Student:

Submission Date: 6 September 2026

Supervisor Approval Page

Title of the Thesis: AdaPruner-KGQA: An Adaptive Search-Space Pruning Framework for Efficient Multi-Hop Knowledge Graph Question Answering

Document Type: B.Sc. Engg. Thesis/Project Plan

Degree Program: B.Sc. Engg. in Computer Science and Engineering

Institution: Department of Computer Science and Engineering, University of Chittagong

Date of Submission: 6 September 2026

Table 1 Evaluation Criteria (to be filled by supervisor(s))

Evaluation Criteria	Select an Option		
Contact hour = 10 hours/week ?	☐ <i>Yes</i>	☐ <i>No</i>	☐ <i>Unknown</i>
Maintained Regular Communication ?	☐ <i>Yes</i>	☐ <i>No</i>	☐ <i>Partly</i>
Maintained Professionalism ?	☐ <i>Yes</i>	☐ <i>No</i>	☐ <i>Partly</i>
Addressed the given comments ?	☐ <i>Yes</i>	☐ <i>No</i>	☐ <i>Partly</i>
Report	☐ <i>Satisfactory</i>	☐ <i>Not Satisfactory</i>	☐ <i>Partly</i>
Approved	☐ <i>Yes</i>	☐ <i>No</i>	

This B.Sc.Engg. Thesis Plan has been reviewed and (NOT) approved by

considering the evaluation criteria outlined in Table 1.

Signature

Dr. Rudra Pratap Deb Nath

Professor

Department of Computer Science and Engineering

University of Chittagong

Declaration

I hereby declare that except where specific reference is made to the work of others, the contents of this dissertation are original and have not been submitted in whole or in part for consideration for any other degree or qualification in this, or any other university. This dissertation is my own work and contains nothing which is the outcome of work done in collaboration with others, except as specified in the text and Acknowledgements. This dissertation contains fewer than 65,000 words including appendices, bibliography, footnotes, tables and equations and has fewer than 150 figures.

Md Sadman Sami Khan

Student ID: 21701081

Session: 2020-2021

Department of Computer Science and Engineering

University of Chittagong

Chattogram-4331, Bangladesh.

Email: sadman.sami@std.cu.ac.bd

I would like to dedicate this thesis to my parents and family for their constant support,
encouragement, and prayers.

Acknowledgements

I would like to express my sincere gratitude to my supervisor Dr. Rudra Pratap Deb Nath for the guidance, support, and valuable suggestions throughout this research. I am also thankful to my teachers and the Department for their support during my academic journey. I would like to thank my family for their constant encouragement, patience, and prayers. Finally, I am grateful to everyone who supported me directly or indirectly in completing this thesis.

Abstract

Multi-hop knowledge graph question answering (KGQA) requires efficient retrieval of reasoning paths from large knowledge graphs. Although Reasoning on Graphs (RoG) constrains retrieval using LLM-generated relation plans, multiple entities may still satisfy the same planned relation, causing residual entity- and path-level branching. This paper introduces AdaPruner-KGQA, which extends RoG with an Adaptive Frontier Pruner (AFP) for selecting relation-valid intermediate branches while preserving the original planner and downstream reasoner. AFP combines lightweight semantic, path-context, structural, and search-progress features with confidence-aware adaptive frontier selection and conservative final-hop protection. Experiments on two benchmark datasets, WebQuestionsSP [9] and ComplexWebQuestions [10], first show that residual search growth remains strongly skewed after relation-level planning, with substantially heavier extreme branching on CWQ. Under the frozen experimental configuration, AdaPruner-KGQA reduces examined-edge search cost by 2.24% on WebQSP and 5.05% on CWQ while retaining 99.20% and 98.72% of RoG-reachable answers, respectively. Final KGQA performance remains close to the reproduced RoG baseline: Hits@1 decreases by 0.4 and 0.5 percentage points, while F1 decreases by 0.2 and 0.5 points on WebQSP and CWQ, respectively. These results show that adaptive intermediate-frontier pruning can reduce residual relation-valid graph exploration while largely preserving answer reachability and downstream QA performance. Our code is available at <https://github.com/sadmantihan/AdaPruner-KGQA>

Contents

1	Introduction	6
2	Preliminaries	10
2.1	Knowledge Graphs and Knowledge Graph Question Answering	10
2.2	Relation Paths and Reasoning Paths	11
2.3	Reasoning on Graphs	12
2.4	Relation-Constrained Graph Retrieval	12
3	Related Work	15
3.1	Semantic Parsing-Based KGQA	15
3.2	Information Retrieval-Based Multi-Hop KGQA	16
3.3	Embedding-Based and Hybrid KGQA	17
3.4	LLM-Based Multi-Hop KGQA	17
3.5	LLM with KG-Based Multi-Hop KGQA	18
3.6	Research Gap	19
4	Methodology	22
4.1	AdaPruner-KGQA Overview	22
4.2	Problem Formulation	24
4.3	Adaptive Frontier Pruner	25
4.3.1	Feature Construction	27
4.3.2	Branch Representation and Scoring	28
4.3.3	Uncertainty-Aware Confidence Estimation	29
4.3.4	Adaptive Frontier Selection	29
4.4	Conservative Final-Hop Strategy	30
4.5	Training Supervision and Objective	33
4.6	Search Complexity and Pruning Overhead Analysis	35

5	Experimental Setup	38
5.1	Datasets	38
5.1.1	WebQuestionsSP	38
5.1.2	ComplexWebQuestions	39
5.2	Comparison Methods	39
5.2.1	Literature-Reported KGQA Methods	40
5.2.2	Frozen Reproduced RoG Baseline	40
5.2.3	Controlled Pruning Baselines	40
5.2.4	Recent Contextual Methods	41
5.3	Implementation and Controlled Experimental Protocol	42
5.3.1	RoG Retrieval Fidelity	42
5.3.2	AFP Training and Hyperparameter Selection	43
5.3.3	Computational Environment and AFP Configuration	44
5.3.4	Search Instrumentation and Counter Semantics	45
5.3.5	Runtime and Reasoner-Cost Measurement	48
5.3.6	Development Oracle and Method-Freezing Protocol	49
5.4	Evaluation Metrics	50
5.5	Research Questions and Experimental Protocol	52
6	Results and Discussion	54
6.1	Baseline Reproduction and Overall KGQA Comparison	54
6.2	RQ1: Search-Space Characterization of RoG	56
6.3	RQ2: Search-Space Reduction through Adaptive Frontier Pruning	58
6.4	RQ3: Answer Retention and KGQA Performance	59
6.5	Efficiency–Effectiveness Trade-Off	61
6.6	Ablation Study	61
6.7	Failure and Case Analysis	67
6.8	Contextual Comparison with Recent Methods	68
6.9	Limitations and Threats to Validity	68
7	Conclusion and Future Work	70
7.1	Conclusion	70
7.2	Future Work	71
	Appendix A	78
	A1 Official Implementation Links of Compared Methods	78

List of Figures

1.1	Local knowledge-graph example for the WebQSP question “What year was LeBron James’ rookie season?”. Under the fixed RoG relation plan <code>player_statistics</code> $\rightarrow$ <code>season</code> , relation-constrained retrieval produces multiple relation-valid entity-level branches. The red branches indicate unnecessary continuations for this question, whereas the green path reaches the gold answer, 2003–04 NBA season, without exploring those branches. This example highlights the branch-selection problem targeted by AdaPrunerKGQA.	7
4.1	Abstract view of the AdaPruner-KGQA architecture. The original RoG relation planner produces the top- K relation plans, which are executed over the knowledge graph from the linked topic entities. During relation-constrained retrieval, AFP operates only on intermediate relation-valid candidate branches before the next frontier is formed. Candidates generated by the final relation bypass pruning. The resulting complete reasoning paths are passed to the unchanged RoG LLM reasoner for answer generation.	23
4.2	Detailed View of Adaptive Frontier Pruner module with 4 components	26
4.3	Illustration of AFP branch-level supervision. Given the relation plan $z = (\text{player_statistics}, \text{season})$ for the question “What year was LeBron James’ rookie season?”, relation-valid candidate branches are labeled according to whether their remaining relation suffix can reach the gold answer. Only the green branch preserving a gold-answer continuation receive positive supervision ($y_i = 1$), while other valid but unproductive branches in red receive negative labels ($y_i = 0$).	34

6.1	Mean Edges Examined for the reproduced RoG baseline and AFP. AFP reduces examined-edge search cost by 2.24% on WebQSP and 5.05% on CWQ.	59
6.2	Search-Space Reduction across the evaluated pruning strategies. SSR is computed from Edges Examined relative to unpruned RoG.	60
6.3	Efficiency–effectiveness trade-off on WebQSP. Higher SSR indicates greater examined-edge reduction, while higher Answer Retention indicates stronger preservation of RoG-reachable answers.	62
6.4	Efficiency–effectiveness trade-off on CWQ. Higher SSR indicates greater examined-edge reduction, while higher Answer Retention indicates stronger preservation of RoG-reachable answers.	63
6.5	Effect of AFP component ablations on WebQSP relative to Full AFP. Positive ΔSSR indicates greater search-space reduction, while positive ΔAR indicates higher Answer Retention.	65
6.6	Effect of AFP component ablations on CWQ relative to Full AFP. Positive ΔSSR indicates greater search-space reduction, while positive ΔAR indicates higher Answer Retention.	66

List of Tables

3.1	Comparison of representative multi-hop KGQA methods across five methodological categories	20
5.1	Dataset splits and maximum relation-plan lengths observed in the frozen RoG artifacts. Plan length refers to planner-generated relations and should not be interpreted as the benchmark’s gold reasoning depth.	39
5.2	Frozen implementation configuration of the AFP module used in the controlled evaluation	45
6.1	Overall KGQA performance on WebQSP and CWQ. Literature-reported values are taken from the original publications and the main RoG comparison. RoG (Reproduced) denotes our frozen reproduction. Best results are represented in bold format.	55
6.2	Frozen reproduced RoG retrieval baseline.	56
6.3	Search-space characterization of unpruned RoG. Values summarize the observed hop-level distributions.	57
6.4	Search-space comparison of RoG and pruning methods. Structural counts are mean values per question. SSR is calculated from total Edges Examined relative to unpruned RoG.	58
6.5	Answer preservation across pruning methods. Answer Retention is computed only over questions reachable by the frozen unpruned RoG baseline.	61
6.6	AFP ablation results	64
1	Official implementation links of compared methods	78

Chapter 1

Introduction

Knowledge graphs (KGs) represent factual knowledge through structured relations between entities and provide an explicit foundation for knowledge-intensive reasoning [41, 11]. Knowledge Graph Question Answering (KGQA) aims to answer natural-language questions by grounding them to entities and relations in a KG and retrieving the corresponding answer entities. For simple questions, the answer can often be obtained by following a single relation from a known entity. For example, the question “Where was LeBron James born?” can be answered by following the relation `birth_place` from the entity `LeBron James` to the city entity `Akron`. However, many real-world questions require combining multiple facts connected through intermediate entities. For instance, answering the question “What year was LeBron James’ rookie season?” requires following `player_statistics` from `LeBron James` to an intermediate `player-statistics` node and then following `season` to reach the corresponding season entity. Hence, the question requires two relational hops before the answer can be obtained. Figure 1.1 illustrates a local KG fragment for this example and shows the two-hop reasoning path leading to the 2003--04 NBA season. As reasoning depth increases, intermediate entities may connect to multiple relation-valid candidates, causing the number of possible reasoning paths to grow and making both accurate retrieval and efficient search challenging in multi-hop KGQA [29, 25].

A central difficulty in multi-hop KGQA is the growth of the graph search space. At each reasoning hop, an intermediate entity may connect to multiple candidate entities, causing the number of possible reasoning paths to expand with reasoning depth and local graph branching. Exhaustive exploration can therefore increase retrieval cost, introduce redundant paths, and expose the downstream reasoner to irrelevant evidence. Retrieval-based methods such as GraftNet and PullNet ad-

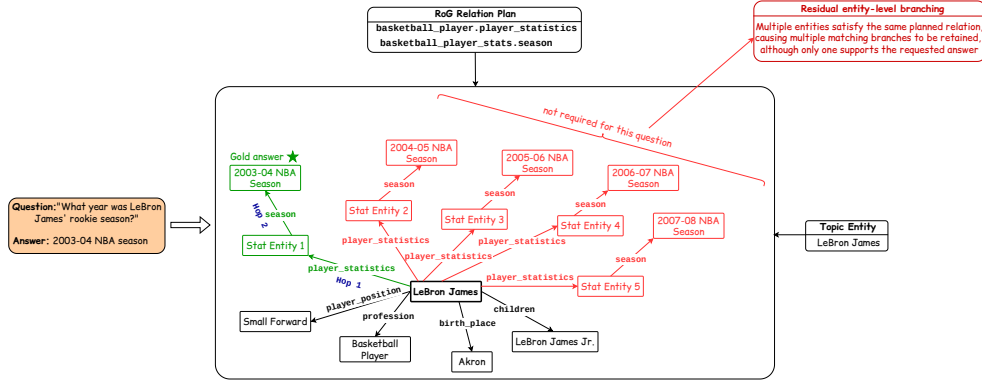


Figure 1.1: Local knowledge-graph example for the WebQSP question “What year was LeBron James’ rookie season?”. Under the fixed RoG relation plan `player_statistics` $\rightarrow$ `season`, relation-constrained retrieval produces multiple relation-valid entity-level branches. The red branches indicate unnecessary continuations for this question, whereas the green path reaches the gold answer, 2003–04 NBA season, without exploring those branches. This example highlights the branch-selection problem targeted by AdaPrunerKGQA.

dress this challenge by constructing question-relevant graph evidence rather than reasoning over the entire KG [5, 6]. More recent approaches further combine learned retrieval with neural or large-language-model reasoning to improve multi-hop KGQA [25, 2].

Reasoning on Graphs (RoG) [1] introduces a planning–retrieval–reasoning framework in which an LLM first predicts question-relevant relation paths. These relation paths serve as reasoning plans for relation-constrained breadth-first search (BFS), after which the retrieved reasoning paths are provided to an LLM reasoner. By constraining graph traversal to predicted relation sequences, RoG provides KG-grounded and interpretable reasoning while avoiding unconstrained relation exploration.

However, relation-level constraints do not necessarily control the number of entity-level branches generated during execution. Even when the relation to be followed at a particular hop is fixed, multiple neighbouring entities may satisfy that relation. Each matching entity creates a new partial reasoning path that may subsequently be expanded. Consequently, relation-guided retrieval can still contain redundant intermediate branches, particularly as reasoning depth and local graph

branching increase. RoG retains relation-matching branches under its generated relation plans without an explicit mechanism for determining which intermediate branches are worth continuing. For the same WebQSP example, RoG generates a relation plan containing `basketball_player.player_statistics` followed by `basketball_player_stats.season`. As illustrated in Figure 1.1, multiple player-statistics branches may satisfy the first planned relation and continue through the second relation toward different season entities. Although these branches are valid under the predicted relation sequence, only the branch leading to the 2003--04 NBA season supports the requested rookie-season answer. Thus, the relation plan determines which relations should be followed, but it does not determine which of the matching entity-level branches are worth continuing. This leaves a residual branch-selection problem inside RoG’s relation-constrained retrieval. Motivated by this limitation, we investigate whether adaptive entity-level frontier pruning can reduce RoG’s residual search space while preserving the answers reachable under its original retrieval process.

In this paper, we propose **AdaPruner-KGQA**, an adaptive search-space pruning framework that extends RoG with an Adaptive Frontier Pruner (AFP) for efficient multi-hop knowledge graph question answering. AdaPruner-KGQA targets the residual entity- and path-level branching that remains during RoG’s relation-constrained retrieval while preserving its original relation planner and downstream reasoner. Specifically, RoG first generates relation paths as reasoning plans and executes them over the KG using relation-constrained BFS. During this traversal, AFP operates only on intermediate candidate branches that already satisfy the required planned relation and scores them for adaptive retention before the next search frontier is formed. The pruner is designed to use lightweight semantic, path-context, and structural information together with confidence-aware selection, retaining more candidates when branch relevance is uncertain and pruning more aggressively when the selection distribution is more confident. A conservative final-hop strategy is used to reduce the risk of directly removing potential answer entities.

The main contributions of this work are threefold. They are condensed as follows-

C1: Residual Search-Space Characterization. We systematically analyze the remaining search space in RoG’s relation-constrained retrieval process by measuring entity- and path-level branching behavior across reasoning hops. The analysis shows that relation-constrained retrieval does not eliminate residual branching and that search cost is strongly skewed, with substantially heavier extreme cases

on CWQ, reaching up to 15,624 candidate branches and 43,473 examined edges.

C2: Adaptive Frontier Pruning Framework. We introduce AdaPruner-KGQA with an Adaptive Frontier Pruner (AFP), a lightweight branch-selection mechanism integrated into RoG’s retrieval stage. Unlike fixed-budget pruning strategies, AFP performs confidence-aware adaptive selection of relation-valid intermediate branches while preserving the original planner, relation constraints, and downstream reasoner. We further conduct feature-family and selector ablations, showing that uncertainty adaptation and temperature scaling have the clearest influence on AFP’s efficiency–retention behavior.

C3: Controlled Efficiency–Effectiveness Evaluation. We conduct a controlled evaluation of AdaPruner-KGQA on WebQuestionsSP (WebQSP) [9] and ComplexWebQuestions (CWQ) [10], keeping the RoG relation plans, knowledge graph, planner, and downstream reasoner unchanged to isolate the effect of adaptive frontier pruning. Compared with reproduced unpruned RoG, AdaPruner reduces examined-edge search cost by 2.24% on WebQSP and 5.05% on CWQ while retaining 99.20% and 98.72% of RoG-reachable answers, respectively. Final KGQA performance remains close to RoG, with Hits@1 decreasing by 0.4 and 0.5 percentage points and F1 decreasing by 0.2 and 0.5 points on WebQSP and CWQ, respectively.

The overall paper is structured as follows. Chapter 2 introduces the preliminaries and relation-constrained retrieval formulation. Related work is reviewed in Chapter 3, including research gap identification and the presentation of research questions with objectives. Chapter 4 provides the description of the AdaPruner-KGQA framework in detail. The experimental setup and evaluation protocol is presented in Chapter 5. Chapter 6 discusses the experimental results, together with search-space reduction, answer preservation, ablation, and failure analyses. Lastly, Chapter 7 concludes the paper and suggests further research direction in future.

Chapter 2

Preliminaries

This chapter introduces the notation and concepts required for the AdaPruner-KGQA framework. It first summarizes the basic formulation of knowledge graphs and multi-hop KGQA, then distinguishes relation paths from entity-level reasoning paths. Finally, it reviews the RoG retrieval process and formalizes relation-constrained graph traversal, including active path prefixes, relation-valid candidate branches, frontier entities, and hop-wise branch expansion.

2.1 Knowledge Graphs and Knowledge Graph Question Answering

A knowledge graph (KG) represents structured knowledge through entities and relations [41]. Formally, a KG is denoted as $G = (\mathcal{E}, \mathcal{R}, \mathcal{T})$, where $\mathcal{E}$ represents the set of entities, $\mathcal{R}$ represents the set of relations, and $\mathcal{T} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$ represents the set of relational triples. Each triple $(e_h, r, e_t) \in \mathcal{T}$ describes a relation $r \in \mathcal{R}$ between a head entity e_h and a tail entity e_t .

Given a natural-language question q , $E_q \subseteq \mathcal{E}$ denotes the set of topic entities linked to the question, while $A_q \subseteq \mathcal{E}$ denotes the corresponding gold answer entity set. The objective of knowledge graph question answering (KGQA) is to identify the correct answer entity or entities in A_q by retrieving and reasoning over relevant information contained in G .

For a single-hop question, an answer may be reached through one relational transition from a topic entity. Multi-hop KGQA, in contrast, requires reasoning through a sequence of intermediate entities and relations before reaching an answer. Con-

sequently, the number of possible reasoning paths may increase with reasoning depth and local graph branching [29].

Although the triple representation is directional, the retrieval experiments in this work follow the exact graph construction and traversal semantics of the reproduced RoG implementation. To avoid assuming purely directed traversal in the formal retrieval model, Section 2.4 introduces a relation-matching operator that captures the neighbor-selection behavior used during RoG execution. Implementation details are provided in Section 5.

2.2 Relation Paths and Reasoning Paths

A *relation path* specifies the sequence of KG relations that should be followed to answer a question. A relation path of length L is defined as

$$z = (r_1, r_2, \dots, r_L), \quad r_i \in \mathcal{R}, \quad (2.1)$$

where r_i denotes the relation to be executed at reasoning hop i .

A *reasoning path* is an entity-level realization of a relation path. For a topic entity $e_0 \in E_q$, a reasoning path corresponding to z can be written as

$$p = e_0 \xrightarrow{r_1} e_1 \xrightarrow{r_2} e_2 \cdots \xrightarrow{r_L} e_L. \quad (2.2)$$

The relation path specifies the relational structure of the reasoning process, whereas a reasoning path specifies the actual sequence of entities encountered while executing that relation sequence over the KG.

Importantly, a single relation path may correspond to multiple reasoning paths. Even when relation r_i is fixed at hop i , several neighboring entities may satisfy that relation. Relation-level guidance therefore does not necessarily determine a unique entity-level continuation.

For later use, we define a partial reasoning path, or path prefix, after h hops as

$$p_h = e_0 \xrightarrow{r_1} e_1 \cdots \xrightarrow{r_h} e_h, \quad 0 \leq h \leq L. \quad (2.3)$$

Here, the subscript h denotes the number of relation transitions represented by the individual path prefix. Such a prefix is considered active only when it is retained in the current frontier for further expansion.

2.3 Reasoning on Graphs

Reasoning on Graphs (RoG) [1] follows a *planning–retrieval–reasoning* architecture for multi-hop KGQA. Given a question q , the planning module first generates a set of candidate relation paths,

$$\mathcal{Z}_q = \{z^{(1)}, z^{(2)}, \dots, z^{(K)}\}, \quad (2.4)$$

where each $z^{(k)}$ represents a possible relational reasoning plan. RoG uses the highest-ranked generated relation paths for subsequent graph retrieval; in its main experimental setting, the top three relation plans are executed.

The retrieval module executes each selected relation path over the KG. Rather than exploring arbitrary relation types from each intermediate entity, retrieval is constrained by the relation required at the corresponding position in the predicted plan. Executing the relation sequence produces KG-grounded reasoning paths that are consistent with the generated relation plan.

Finally, the retrieved reasoning paths, together with the original question, are provided to the downstream language-model reasoner, which generates the final answer.

In AdaPruner-KGQA, the RoG relation planner and downstream reasoner remain unchanged. The proposed modification is introduced only within the relation-constrained graph retrieval stage.

2.4 Relation-Constrained Graph Retrieval

Consider a relation plan, $z = (r_1, r_2, \dots, r_L)$ and a topic entity $e_0 \in E_q$. RoG executes the relation sequence using relation-constrained breadth-first traversal.

Let $\mathcal{P}_h$ denote the set of active path prefixes after h relation transitions, where $p_{(h)} \in \mathcal{P}_h$ denotes an individual prefix retained for further expansion. Initially, $\mathcal{P}_0 = \{(e_0)\}$. For an active path prefix $p \in \mathcal{P}_h$, let $last(p) \in \mathcal{E}$ denote its terminal entity. In particular, for $p_{(h)} = e_0 \xrightarrow{r_1} \dots \xrightarrow{r_h} e_h$, $last(p_{(h)}) = e_h$. To represent the implementation-level relation matching used during retrieval, define

$$\mathcal{M}(e, r) = \{e' \in \mathcal{E} \mid e' \text{ matches relation } r \text{ from } e\}. \quad (2.5)$$

Here, $\mathcal{M}(e, r)$ denotes the set of neighbouring entities accepted by the reproduced RoG retrieval procedure when relation r is executed from entity e . The operator

abstracts the implementation-specific graph construction and relation-matching behavior, including the undirected graph representation used in our reproduced RoG setup.

At hop $h + 1$, the traversal examines the neighborhood of $\text{last}(p)$ and generates only extensions that satisfy the next required relation r_{h+1} . The set of relation-matching candidate extensions ($\mathcal{C}_h$) is therefore,

$$\mathcal{C}_h = \{p \circ (r_{h+1}, e') \mid p \in \mathcal{P}_h, e' \in \mathcal{M}(\text{last}(p), r_{h+1})\}, \quad (2.6)$$

where $\circ$ denotes sequential path extension. Specifically, for $p_{(h)} = e_0 \xrightarrow{r_1} e_1 \cdots \xrightarrow{r_h} e_h$, the extension $p_{(h)} \circ (r_{h+1}, e')$ appends the relation–entity pair (r_{h+1}, e') to the existing prefix.

In the unpruned RoG traversal, every valid relation-matching candidate is admitted to the next set of active prefixes ($\mathcal{P}_{h+1}^{\text{RoG}}$)-

$$\mathcal{P}_{h+1}^{\text{RoG}} = \mathcal{C}_h. \quad (2.7)$$

The relation constraint removes branches associated with relation types that do not match the current plan. However, it does not prevent several entities from satisfying the same required relation. Consequently, a single active prefix may generate multiple valid candidate extensions.

It is important to distinguish the number of active path prefixes from the number of unique entities represented in the frontier. We, therefore define the set of unique frontier entities ($\mathcal{U}_h$) after hop h as,

$$\mathcal{U}_h = \{\text{last}(p) \mid p \in \mathcal{P}_h\}. \quad (2.8)$$

Accordingly, $|\mathcal{P}_h|$ denotes the number of active reasoning branches, whereas $|\mathcal{U}_h|$ denotes the number of unique frontier entities. These quantities need not be equal because multiple distinct path prefixes may terminate at the same entity.

To characterize hop-wise branch expansion, we define the relation-valid branch expansion ratio (ρ_h) as

$$\rho_h = \frac{|\mathcal{C}_h|}{|\mathcal{P}_h|}, \quad |\mathcal{P}_h| > 0 \quad \begin{cases} \rho_h > 1, & \text{frontier expansion} \\ \rho_h = 1, & \text{unchanged frontier size} \\ 0 \leq \rho_h < 1, & \text{frontier contraction} \end{cases} \quad (2.9)$$

The ratio is defined only for non-empty active frontiers; if $\mathcal{P}_h = \emptyset$, traversal of the corresponding relation plan terminates. Since $\mathcal{P}_{h+1}^{\text{RoG}} = \mathcal{C}_h$, the ratio equivalently

measures the multiplicative change in the number of active path prefixes between consecutive hops in the unpruned RoG. Repeated values of $\rho_h > 1$ across successive hops indicate that relation-valid reasoning branches can continue to grow even though the relation sequence itself remains fixed.

Candidate-branch count alone does not fully capture retrieval cost, since relation-constrained traversal must inspect adjacency entries before identifying those that satisfy the required relation. We therefore characterize the search space using complementary quantities: active path prefixes ($|\mathcal{P}_h|$), unique expanded entities ($|\mathcal{X}_h|$), examined edges (E_h), relation-valid candidate branches ($|\mathcal{C}_h|$), unique frontier entities ($|\mathcal{U}_h|$), retrieved reasoning paths ($|\mathcal{P}_q^{\text{ret}}|$), and peak frontier size (F_q^{max}). Together, these measures distinguish path-level branching, unique-entity exploration, and actual graph-traversal work.

The resulting entity and path-level branching constitutes the residual search space targeted by AFP. AdaPruner-KGQA therefore operates on the relation-valid candidate set $\mathcal{C}_h$ and selects which intermediate path prefixes are admitted to the next active frontier, without modifying the RoG relation plan. This branch-selection mechanism is formalized in Section 4.

Chapter 3

Related Work

Multi-hop KGQA has been approached through several methodological paradigms, including semantic parsing, information retrieval, embedding-based or hybrid reasoning, as well as more recent LLM-based and LLM with KG-based graph reasoning approaches [29, 1]. These paradigms differ not only in how answers are predicted, but also in how they control the potentially large search space induced by multi-hop reasoning. Since the present work focuses on reducing graph exploration, we review representative methods primarily from the perspective of their search and retrieval strategies, together with the remaining limitations relevant to search-space control. Finally, we synthesize these observations to identify the research gap and explain how our approach addresses residual entity- and path-level branching within relation-constrained retrieval.

3.1 Semantic Parsing-Based KGQA

Semantic parsing-based KGQA maps a natural-language question into an executable symbolic representation, such as a logical form, query graph, or SPARQL program. Early work by Berant et al. [19] learned logical forms from question-answer pairs, while Yih et al. [8] introduced staged query-graph generation for Freebase question answering. Later approaches improve complex query construction through ranking, constrained generation, or dynamic program induction. ReTraCk [20], ArcaneQA [21], and RnG-KBQA [38], for example, introduce increasingly flexible mechanisms for constructing and selecting executable query structures.

The principal advantage of semantic parsing is that the resulting reasoning process

is explicit and interpretable. Search is controlled by restricting reasoning to generated logical structures rather than traversing the entire KG. However, complex questions may require generating or ranking many candidate query graphs, and errors in entity linking, relation linking, schema alignment, or intermediate program construction can propagate to the final answer. These limitations motivate retrieval-oriented approaches that operate more directly over graph evidence.

3.2 Information Retrieval-Based Multi-Hop KGQA

Information retrieval-based KGQA first extracts a question-specific subgraph, set of entities, or collection of reasoning paths and subsequently performs answer reasoning over the retrieved evidence. GraftNet [5] constructs heterogeneous question-specific graphs and performs graph-based reasoning, while PullNet [6] iteratively expands the retrieved graph according to the current reasoning state. Subsequent methods place greater emphasis on learning which graph evidence should be retrieved. The subgraph-retrieval approach of Zhang et al. [39] retrieves reasoning evidence before downstream QA, whereas UniKGQA [25] unifies retrieval and reasoning within a common framework.

Recent retrieval architectures increasingly target both retrieval quality and search-space efficiency. PipeNet [32] introduces semantic pruning over knowledge graph structures to remove irrelevant concepts and reduce noisy evidence before reasoning. SubgraphRAG [2] further combines lightweight retrieval with structural information to score KG triples and construct flexible question-specific subgraphs for downstream LLM reasoning. GNN-RAG [12] instead uses graph neural representations to identify likely answer entities and extracts supporting paths connecting them to the question entities before LLM-based reasoning.

These approaches demonstrate that reducing irrelevant graph evidence can improve retrieval efficiency and reasoning quality. However, their pruning or selection mechanisms primarily operate on retrieved subgraphs, triples, or external evidence collections before the final reasoning stage. They do not specifically address the residual entity-level branching that emerges during execution of an already predicted relation path. In contrast, AdaPruner-KGQA focuses on adaptive frontier pruning inside a relation-constrained traversal process, where multiple entities may satisfy the same planned relation and create unnecessary intermediate reasoning branches.

3.3 Embedding-Based and Hybrid KGQA

Embedding-based approaches represent entities, relations, questions, or reasoning states in continuous vector spaces and use learned representations to score candidate answers or reasoning transitions. KV-Mem [14] uses memory addressing over knowledge facts, while EmbedKGQA [4] combines question representations with knowledge-base embeddings to rank candidate answer entities. Rce-KGQA [7] further models relational chains to improve multi-hop reasoning over ordered relation sequences. Moving beyond conventional entity-specific knowledge graph embedding (KGE) scoring, KGT5 [18] formulates both knowledge graph completion and KGQA as sequence-to-sequence tasks, using a single encoder-decoder Transformer pretrained on link prediction and subsequently fine-tuned for question answering. Its autoregressive decoding strategy avoids explicitly scoring all entities in the KG during inference.

Hybrid approaches additionally combine learned representations with explicit graph propagation or structured reasoning. TransferNet [34] performs differentiable score transfer over a relation graph, while ReaRev [33] uses adaptive iterative graph reasoning to revise question representations and propagate information through relevant graph structures.

Embedding and hybrid approaches provide learned mechanisms for representing relational information and reasoning over candidate answers or graph structures. However, their primary objectives are answer prediction, representation learning, or graph-based reasoning rather than explicitly controlling residual entity-level branching within a RoG-style relation-constrained traversal. Consequently, they do not directly address the problem of deciding which relation-matching intermediate branches should continue during retrieval.

3.4 LLM-Based Multi-Hop KGQA

The emergence of large language models (LLMs) has introduced a new direction for multi-hop question answering. Models such as Flan-T5-xl [23], Alpaca [30], LLaMA2-Chat [13], and ChatGPT [16] can answer knowledge-intensive questions directly through zero-shot or few-shot prompting without explicitly constructing symbolic queries or traversing a knowledge graph. Chain-of-thought (CoT) prompting further improves multi-step reasoning by encouraging the model to generate intermediate reasoning steps before producing an answer in ChatGPT-CoT [17].

Despite their strong language understanding and reasoning capability, standalone LLMs rely primarily on knowledge encoded in their parameters. Consequently, they may generate unsupported facts, lack access to updated knowledge, or produce reasoning steps that cannot be verified against an external knowledge source. These limitations have motivated approaches that explicitly integrate LLM reasoning with structured knowledge graphs.

3.5 LLM with KG-Based Multi-Hop KGQA

Recent approaches increasingly combine large language models with knowledge graphs to ground planning, retrieval, and reasoning in structured factual knowledge. KG-GPT [28] integrates KG facts with LLM-based reasoning, while Think-on-Graph [35] enables iterative exploration of relevant graph evidence through LLM-guided reasoning. DECAF [22] retrieves relevant knowledge from a linearized knowledge base and jointly generates answers and logical forms for knowledge-grounded question answering. KG-CoT [40] incorporates KG-grounded reasoning paths into chain-of-thought prompting, whereas Plan-on-Graph [42] performs adaptive and self-correcting planning over graph structures.

Recent works have further investigated adaptive knowledge retrieval to improve the efficiency of LLM-based KG reasoning. AMAR [31] introduces adaptive multi-aspect retrieval-augmentation by selecting relevant KG entities, relations, and subgraph information before providing knowledge context to the LLM. Similarly, retrieval-oriented approaches aim to reduce irrelevant evidence presented to the reasoning module. However, these methods primarily optimize the construction or selection of retrieved knowledge and do not address redundant intermediate branches generated during graph traversal after a reasoning plan has already been established.

Reasoning on Graphs (RoG) [1] further separates multi-hop KGQA into planning, retrieval, and reasoning. An LLM first generates relation paths as reasoning plans, which are executed over the KG using relation-constrained breadth-first search before the retrieved reasoning paths are provided to the downstream LLM reasoner. Although this restricts relation-level exploration, multiple entities may still satisfy the same planned relation, leaving residual entity- and path-level branching during retrieval.

AdaPruner-KGQA addresses this remaining search space by introducing the Adaptive Frontier Pruner (AFP) within RoG’s retrieval stage. The framework retains

RoG’s LLM-based relation planner and downstream reasoner, while AFP operates only on relation-matching intermediate branches generated during KG traversal. Therefore, AdaPruner-KGQA remains an LLM with KG-based KGQA framework, but its contribution differs from existing adaptive retrieval approaches by performing entity-level branch selection inside a relation-constrained reasoning process.

Table 3.1 presents a comparative overview of representative methods across the five KGQA paradigms considered in this study, focusing on four methodological features related to search-space control and retrieval efficiency. Branching Control indicates whether a method explicitly limits the growth of intermediate entities, reasoning paths, or search states during multi-hop reasoning. Selective Retrieval denotes the use of retrieval, ranking, or filtering mechanisms to prioritize relevant entities, triples, subgraphs, paths, or other evidence while reducing unnecessary retrieval. Efficient Inference indicates whether the method incorporates an explicit mechanism intended to reduce or control graph-processing, multi-stage, or LLM inference cost. Finally, Explicit Pruning denotes the presence of a dedicated mechanism for removing candidate entities, relations, paths, branches, or search states during retrieval or reasoning. Retrieval or ranking alone is not considered explicit pruning. Code Availability is also reported to indicate whether a publicly accessible implementation of each method is available and all the verified links are provided in Appendix A1.

3.6 Research Gap

To identify the research gap, we review representative multi-hop KGQA methods across semantic parsing, information retrieval, embedding/hybrid, and LLM-based paradigms, with particular attention to how graph search and retrieval are controlled [29, 2, 12].

RoG [1] constrains graph traversal through predicted relation paths, but multiple entities may still satisfy the same planned relation and generate additional intermediate reasoning branches. Although recent methods such as MoKGR [3] investigate query-adaptive pruning in other knowledge-graph reasoning settings, their task formulation, datasets, and reasoning architecture differ from the RoG-style KGQA retrieval setting considered in this work. Accordingly, lightweight adaptive pruning of entity-level branches within an already fixed RoG relation plan remains underexplored. Based on this gap, we investigate the following research questions-

Table 3.1: Comparison of representative multi-hop KGQA methods across five methodological categories

Model	Dataset(s)	Search / Reasoning Control	Features				Code Availability
			Branching Control	Selective Retrieval	Efficient Inference	Explicit Pruning	
Semantic Parsing-Based KGQA							
Berant et al. [19]	WebQuestions	Logical-form generation	×	✓	✓	×	Yes
Yih et al. [8]	WebQSP	Query-graph generation	×	✓	✓	✓	Yes
ArcaneQA [21]	WebQSP, GrailQA	Dynamic program induction	×	✓	×	✓	Yes
RnG-KBQA [38]	WebQSP, GrailQA	Rank-then-generate	×	✓	×	×	Yes
Information Retrieval-Based Multi-Hop KGQA							
GraftNet [5]	WikiMovies, WebQuestions	Subgraph retrieval	✓	×	×	×	Yes
PullNet [6]	WikiMovies, WebQSP, CWQ	Iterative expansion	×	×	×	×	Yes
UniKGQA [25]	WebQSP, CWQ	Unified retrieval–reasoning	✓	×	×	×	Yes
SR+NSM [39]	WebQSP, CWQ	Subgraph retrieval	✓	×	×	×	Yes
PipeNet [32]	CommonsenseQA, OpenBookQA	Semantic node pruning	✓	✓	×	✓	Yes
SubgraphRAG [2]	WebQSP, CWQ	Top- <i>K</i> triple retrieval	✓	×	×	×	Yes
GNN-RAG [12]	WebQSP, CWQ	GNN-guided path retrieval	✓	×	×	×	Yes
Embedding-Based and Hybrid KGQA							
KV-Mem [14]	WikiMovies, WikiQA	Memory addressing	✓	✓	×	×	Yes
EmbedKGQA [4]	WebQSP, WikiMovies	Embedding-based ranking	✓	✓	×	✓	Yes
NSM [15]	WikiMovies, WebQSP, CWQ	Neural state propagation	✓	×	×	×	Yes
TransferNet [34]	WebQSP, MetaQA	Differentiable transfer	✓	✓	×	×	Yes
KGT5 [18]	MetaQA, WebQSP, CWQ	Seq2seq answer generation	✓	✓	×	×	Yes
Rce-KGQA [7]	WikiMovies, WebQSP, CWQ	Relational-chain reasoning	✓	✓	×	×	Yes
ReaRev [33]	WebQSP, CWQ	Adaptive graph reasoning	✓	×	×	×	Yes
LLM-Based Multi-Hop KGQA							
LLaMA2-Chat-7B [13]	WebQSP, CWQ	Direct generation	✓	✓	×	×	Yes
ChatGPT [16]	WebQSP, CWQ	Zero-shot reasoning	✓	✓	×	×	Yes
ChatGPT+CoT [17]	WebQSP, CWQ	Chain-of-thought prompting	✓	✓	×	×	Yes
LLM with KG-Based Multi-Hop KGQA							
KG-GPT [28]	Multiple KG benchmarks	KG-guided LLM reasoning	✓	×	×	×	Yes
DECAF [22]	WebQSP	Answer-form generation	✓	✓	×	×	Yes
Think-on-Graph [35]	WebQSP, CWQ	LLM-guided traversal	×	✓	×	✓	Yes
Plan-on-Graph [42]	CWQ, WebQSP, GrailQA	Adaptive graph planning	×	✓	×	✓	Yes
RoG [1]	WebQSP, CWQ	Relation-constrained BFS	×	×	×	×	Yes
AMAR [31]	WebQSP, CWQ	Adaptive multi-aspect retrieval	✓	✓	×	×	Yes
AdaPruner-KGQA	WebQSP, CWQ	Adaptive frontier pruning	✓	✓	✓	✓	Yes

Note: ✓ indicates that the corresponding feature is explicitly supported by the method, whereas × indicates that the feature is absent or not a primary component of the method.

RQ1: Where and to what extent does search-space growth occur during RoG’s relation-constrained graph retrieval?

Way to solve: We profile RoG’s retrieval process on WebQSP [9] and CWQ [10] datasets at each reasoning hop. We measure active path prefixes, expanded entities, examined edges, candidate branches, frontier size, retrieved reasoning paths, and answer reachability. The relation-valid branch expansion ratio (ρ_h) is also used to show where the frontier grows or contracts. These measurements identify the hops and datasets in which residual search-space growth is most significant. This directly maps to Contribution C1 (Residual Search-Space Characterization) outlined in Section 1, as it establishes where residual entity- and path-level branching persists within RoG’s relation-constrained retrieval.

RQ2: Can adaptive frontier pruning reduce unnecessary graph exploration more effectively than unpruned RoG and simple fixed-pruning strategies?

Way to solve: We apply AFP after RoG generates the relation-valid candidate set (C_h). AFP scores intermediate candidate branches using lightweight features and adaptively selects which branches are retained for the next hop. Its search reduction is compared with unpruned RoG, fixed Top- B , and fixed-threshold pruning under the same relation plans and graph settings, primarily using examined-edge reduction and frontier-level search measures. An ablation study is further conducted to assess the contribution of the main AFP components to search-space reduction. It maps to Contribution C2 (Adaptive Frontier Pruning Framework) outlined in Section 1, as it evaluates whether confidence-aware adaptive frontier selection provides more effective search control than unpruned and fixed-pruning alternatives.

RQ3: Can the reduced search space preserve RoG-reachable answers and final KGQA performance while improving retrieval efficiency?

Way to solve: We evaluate the frozen AFP model against unpruned RoG using the same downstream reasoner. Answer preservation is measured using answer retention and retrieval coverage, while final KGQA performance is measured using Hits@1 and F1. Efficiency is evaluated through examined edges, frontier size, retrieval latency, pruning overhead, and reasoner-input size. We also analyze pruning failures to identify when answer-supporting branches are removed. This maps to Contribution C3 (Controlled Efficiency–Effectiveness Evaluation) outlined in Section 1, as it assesses whether the achieved search-space reduction can preserve answer reachability and final KGQA effectiveness under the controlled RoG-based evaluation setting.

Chapter 4

Methodology

This chapter presents the methodology of AdaPruner-KGQA and formalizes how adaptive frontier pruning is integrated into RoG’s relation-constrained retrieval process. It first provides the framework overview, defines the pruning problem and the operation of the Adaptive Frontier Pruner (AFP), followed by the branch representation, scoring, and confidence-aware frontier selection strategy. The section then describes the conservative final-hop policy, the complete retrieval algorithm, the training supervision and objective, and the search complexity and pruning overhead considered in our approach.

4.1 AdaPruner-KGQA Overview

AdaPruner-KGQA extends the relation-constrained graph-retrieval stage of Reasoning on Graphs (RoG) while preserving its original relation planner and downstream LLM reasoner. Given a natural-language question q , the RoG relation planner first generates a ranked set of relation plans $\mathcal{Z}_q$, as defined in Eq. (2.4). Figure 4.1 depicts the abstract view of this approach and illustrates how adaptive frontier pruning is integrated into RoG’s relation-constrained retrieval process. These plans specify which relations should be followed during graph retrieval. The linked topic entities E_q and the knowledge graph G provide the corresponding retrieval inputs. AdaPruner-KGQA does not modify these inputs or regenerate the relation plans; instead, it intervenes only during the execution of each fixed relation plan.

For a relation plan $z = (r_1, r_2, \dots, r_L) \in \mathcal{Z}_q$, retrieval proceeds according to the next planned relation r_{h+1} at hop $h + 1$. Using the notation introduced in Sec-

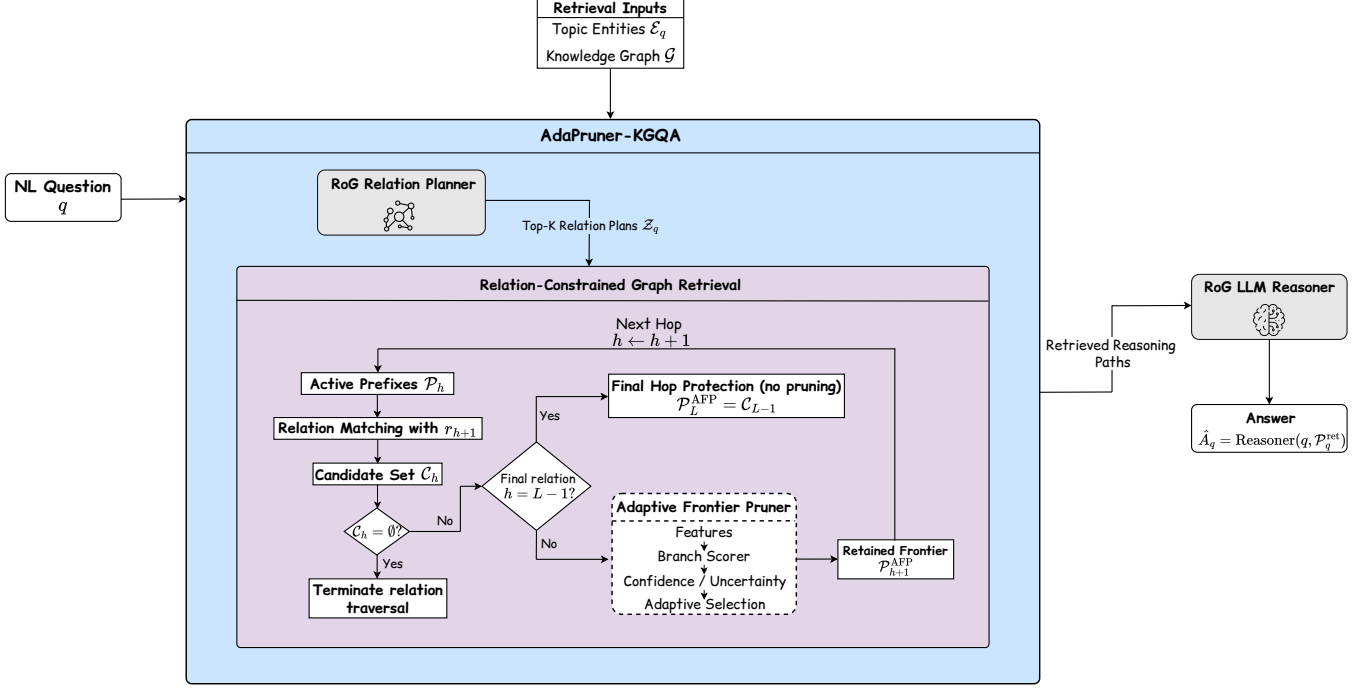


Figure 4.1: Abstract view of the AdaPruner-KGQA architecture. The original RoG relation planner produces the top- K relation plans, which are executed over the knowledge graph from the linked topic entities. During relation-constrained retrieval, AFP operates only on intermediate relation-valid candidate branches before the next frontier is formed. Candidates generated by the final relation bypass pruning. The resulting complete reasoning paths are passed to the unchanged RoG LLM reasoner for answer generation.

tion 2.4, the current active path prefixes $\mathcal{P}_h$ are expanded through r_{h+1} to produce the relation-valid candidate set $\mathcal{C}_h$. We use $\mathcal{P}_{h+1}^{\text{AFP}}$ to denote the frontier retained by the Adaptive Frontier Pruner (AFP) for the next hop. $\mathcal{P}_q^{\text{ret}}$ is denoted as the set of complete reasoning paths retrieved for question q across all executed topic entities and selected relation plans. A path is included in $\mathcal{P}_q^{\text{ret}}$ only when it successfully follows all relations of its corresponding relation plan. After all selected relation plans and topic entities have been processed, the complete reasoning paths retrieved for question q are collected in $\mathcal{P}_q^{\text{ret}}$. The final answer produced by the unchanged RoG reasoner is denoted by $\hat{A}_q$.

The overall abstract execution flow of AdaPruner-KGQA is represented in Figure 4.1. Starting from the left, the question q is provided to the RoG relation planner, which generates the top- K relation plans $\mathcal{Z}_q$. Together with the topic entities E_q and knowledge graph G , these plans are passed to the relation-constrained retrieval stage. For a relation plan, the current active prefixes $\mathcal{P}_h$ are expanded through the next planned relation r_{h+1} using RoG’s original relation-matching procedure, producing the relation-valid candidate set $\mathcal{C}_h$. If $\mathcal{C}_h = \emptyset$, the current topic-entity–relation-plan traversal terminates because no valid continuation exists, while other topic entities and relation plans may still be processed. If the current transition is the final relation, i.e., $h = L - 1$, AFP is bypassed and all candidates are retained: $\mathcal{P}_L^{\text{AFP}} = \mathcal{C}_{L-1}$. Here, $\mathcal{P}_L^{\text{AFP}}$ contains the complete reasoning paths obtained after the final relation. This no-pruning policy at the answer-producing hop is termed *Final-Hop Protection*. For an intermediate hop, $h < L - 1$, $\mathcal{C}_h$ is passed to the Adaptive Frontier Pruner shown in the dashed block of Figure 4.1. This block constructs lightweight branch features, scores the candidates, estimates their confidence or uncertainty, and adaptively selects the branches to retain. The selected candidates form $\mathcal{P}_{h+1}^{\text{AFP}}$, which becomes the active frontier for the next hop. Retrieval then continues with the next relation of the same fixed RoG plan, while discarded branches are not expanded further.

After the full relation plan is successfully executed, the resulting complete reasoning paths are added to $\mathcal{P}_q^{\text{ret}}$. This process is repeated across the selected relation plans and topic entities, after which the retrieved paths are supplied to the unchanged RoG LLM reasoner. The final answer is obtained as

$$\hat{A}_q = \text{Reasoner}(q, \mathcal{P}_q^{\text{ret}})$$

The figure therefore also establishes the methodological boundary of AdaPruner-KGQA: AFP neither predicts or modifies relation plans nor replaces the downstream reasoner; it only selects which already relation-valid intermediate branches are propagated to the next retrieval frontier.

4.2 Problem Formulation

Consider a fixed RoG relation plan $z = (r_1, \dots, r_L) \in \mathcal{Z}_q$ executed for question q from a linked topic entity. As defined in Section 2.4, the current active prefixes $\mathcal{P}_h$ are expanded according to the next planned relation r_{h+1} , producing the relation-valid candidate set $\mathcal{C}_h$ at hop h .

The pruning problem arises at an intermediate hop when $|\mathcal{C}_h| > 1$. Although every candidate in $\mathcal{C}_h$ satisfies the same relation constraint imposed by the fixed RoG plan, these candidates need not be equally useful for reaching an answer. Under unpruned RoG, all such branches are propagated, so unnecessary intermediate branches may trigger additional graph exploration at subsequent hops.

AdaPruner-KGQA therefore formulates retrieval at each intermediate hop as a branch-selection problem. Given the question q , fixed relation plan z , current hop h , and relation-valid candidate set $\mathcal{C}_h$, AFP learns a selection operator $\mathcal{S}_\theta$ such that,

$$\mathcal{P}_{h+1}^{\text{AFP}} = \mathcal{S}_\theta(q, z, h, \mathcal{C}_h) \subseteq \mathcal{C}_h, \quad 0 \leq h < L - 1, \quad (4.1)$$

where θ denotes the learnable parameters of the branch-scoring model and $\mathcal{P}_{h+1}^{\text{AFP}}$ is the retained frontier propagated to the next hop. The central objective is therefore to retain a compact set of promising relation-valid branches that reduces subsequent graph exploration while preserving, as much as possible, the answer-supporting continuations already reachable under the original RoG retrieval process. The problem is thus not to predict a new relation or revise the RoG plan, but to determine **which branches satisfying the planned relation should continue**. AFP cannot recover an answer when the required relation plan or supporting graph path is absent from the original RoG retrieval space.

4.3 Adaptive Frontier Pruner

Adaptive Frontier Pruner (AFP) is a lightweight branch-level pruning module designed to reduce unnecessary reasoning expansion while preserving answer-supporting paths. Although RoG [1] effectively reduces the search space by constraining graph exploration according to LLM-generated relation plans, each relation transition can still produce a large number of valid candidate branches. These relation-valid candidates are not equally useful: many branches satisfy the required relation constraint but cannot contribute to the final answer. Expanding such branches increases the intermediate search frontier and causes unnecessary graph traversal in later hops. Therefore, an effective KGQA system requires not only relation-constrained retrieval but also an adaptive mechanism that can identify and preserve the most promising reasoning frontiers.

To address this challenge, we introduce this module to mitigate intermediate reasoning branch explosion by selectively preserving promising candidate paths whose detailed view is represented in Figure 4.2. Given the candidate branches

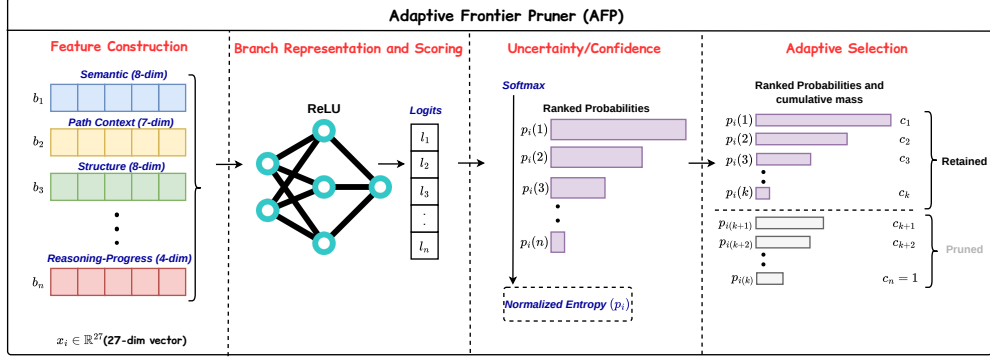


Figure 4.2: Detailed View of Adaptive Frontier Pruner module with 4 components

generated by the original RoG retrieval process, AFP estimates the future usefulness of each branch and dynamically selects a compact frontier for subsequent exploration. In this way, AFP transforms retrieval from a fixed expansion process into an adaptive search procedure where the exploration budget is adjusted according to the quality and uncertainty of the current reasoning frontier.

Formally, at hop h , RoG [1] generates a set of relation-valid candidate branches $\mathcal{C}_h$ according to the planned relation sequence. AFP operates on this candidate set and produces a reduced frontier $\mathcal{P}_{h+1}^{\text{AFP}}$.

$$\mathcal{P}_{h+1}^{\text{AFP}} = \text{AFP}(\mathcal{C}_h), \quad (4.2)$$

where AFP does not modify the generated relation plan or introduce additional graph exploration. Instead, it determines which existing candidates should be retained before the next expansion step. Therefore, AFP complements RoG by addressing the intermediate search explosion problem while maintaining the original reasoning constraints.

The central objective of AFP is to maximize the probability of retaining answer-supporting reasoning paths while minimizing unnecessary frontier growth. This requires balancing two competing goals: aggressive pruning to reduce computational overhead and conservative retention to avoid removing potentially correct paths. To achieve this balance, AFP performs four sequential components on each candidate frontier, which are briefly described in subsections 4.3.1 to 4.3.4.

O1: Feature Construction. Each candidate branch is represented using lightweight

semantic, path-context, structural, and reasoning-progress features available during retrieval.

O2: Branch Representation and Scoring. The extracted features are transformed into branch representations, and a neural scorer estimates the relevance of each candidate continuation.

O3: Uncertainty/Confidence Estimation. The score distribution of the current frontier is analyzed to estimate whether the pruning decision is confident or ambiguous.

O4: Adaptive Frontier Selection. The retained frontier size is dynamically determined based on the confidence of the current decision instead of applying a fixed pruning threshold.

4.3.1 Feature Construction

Given the candidate frontier $\mathcal{C}_h$ generated at hop h , each candidate branch is represented as $b_i \in \mathcal{C}_h$. In the Figure 4.2, AFP constructs a feature representation for each branch to capture whether the candidate remains useful for future reasoning. We denote the resulting feature vector of branch b_i as $\mathbf{x}_i$, which is used as the input representation for the branch scoring module. Specifically, $\mathbf{x}_i$ is constructed by concatenating four complementary feature groups-

$$\mathbf{x}_i = [\mathbf{f}_i^{sem} \parallel \mathbf{f}_i^{path} \parallel \mathbf{f}_i^{struct} \parallel \mathbf{f}_i^{prog}], \quad (4.3)$$

where $\parallel$ denotes vector concatenation and $\mathbf{x}_i \in \mathbb{R}^{27}$ represents the final 27-dimensional branch feature vector. The representation consists of an 8-dimensional semantic feature $\mathbf{f}_i^{sem} \in \mathbb{R}^8$, a 7-dimensional path-context feature $\mathbf{f}_i^{path} \in \mathbb{R}^7$, an 8-dimensional structural feature $\mathbf{f}_i^{struct} \in \mathbb{R}^8$, and a 4-dimensional reasoning-progress feature $\mathbf{f}_i^{prog} \in \mathbb{R}^4$.

The semantic feature $\mathbf{f}_i^{sem}$ captures the compatibility between the candidate branch and the question intent, enabling AFP to identify branches that are more relevant to the required answer. The path-context feature $\mathbf{f}_i^{path}$ encodes information from previously traversed relations and entities, helping maintain consistency with the existing reasoning trajectory. The structural feature $\mathbf{f}_i^{struct}$ represents lightweight graph characteristics available during retrieval without requiring additional graph exploration. Finally, the reasoning-progress feature $\mathbf{f}_i^{prog}$ captures the current position within the multi-hop reasoning process, allowing AFP to adapt its pruning decision according to the remaining search depth.

For the Figure 1.1 example, “*What year was LeBron James’ rookie season?*”, RoG [1] may generate multiple relation-valid branches after expanding the entity corresponding to LeBron James. AFP does not immediately remove these candidates; instead, each branch is first transformed into the 27-dimensional representation described above. This allows the pruner to distinguish between branches that are likely to support the answer and branches that are valid graph transitions but less useful for subsequent reasoning.

4.3.2 Branch Representation and Scoring

Let the branch representation be denoted as $\mathbf{x}_i$. AFP estimates the relevance of each candidate branch using a lightweight feed-forward scoring network. The first layer transforms the input representation into a hidden branch state-

$$\mathbf{h}_i = \text{ReLU}(W_1 \mathbf{x}_i + b_1), \quad (4.4)$$

where $\mathbf{h}_i$ denotes the hidden representation, W_1 and b_1 are learnable weight and bias parameters, respectively, and $\text{ReLU}(\cdot)$ is the rectified linear unit activation function-

$$\text{ReLU}(z) = \begin{cases} z, & \text{if } z > 0, \\ 0, & \text{if } z \leq 0. \end{cases} \quad (4.5)$$

The hidden representation is then mapped to a scalar relevance logit to measure the branch-level relevance before probability normalization which is visualized in Figure 4.2. $\ell_i = W_2 \mathbf{h}_i + b_2$, where ℓ_i represents the branch’s learned relevance before sigmoid conversion, W_2 and b_2 denote the second-layer learnable parameters. The branch confidence score is obtained by applying a sigmoid function s_i -

$$s_i = \sigma(\ell_i) = \frac{1}{1 + \exp(-\ell_i)} \quad (4.6)$$

where $s_i \in (0, 1)$ represents the estimated likelihood that candidate branch b_i remains useful for successful reasoning. The sigmoid function is used to transform the unbounded relevance logit into a normalized confidence score, allowing AFP to interpret the output as a branch-level relevance probability for supervision and confidence estimation. In the Figure 1.1 example, branches corresponding to rookie-season-related information receive higher relevance scores than other relation-valid but less useful branches, such as team affiliation or player attributes. However, AFP does not prune solely based on individual branch scores; instead, it considers the relative confidence distribution among all candidates in the current frontier, which enables uncertainty-aware adaptive selection.

4.3.3 Uncertainty-Aware Confidence Estimation

To determine how aggressively the frontier should be reduced, AFP estimates the uncertainty probabilistic value of the current candidate distribution. In the Figure 4.2, the branch logits are normalized using a temperature-scaled softmax function,

$$\pi_i = \frac{\exp(\ell_i/T)}{\sum_{j=1}^{|\mathcal{C}_h|} \exp(\ell_j/T)}, \quad T > 0, \quad (4.7)$$

where T is a temperature parameter controlling the sharpness of the normalized branch-probability distribution, and π_i represents the relative confidence of candidate branch b_i within the current frontier. When $T < 1$, the distribution becomes sharper, increasing the separation between high- and low-logit branches; consequently, smaller values of T make the highest-scoring branches more dominant. In contrast, when $T > 1$, the distribution becomes flatter, assigning more similar probability mass to the candidate branches. The uncertainty of the frontier is measured using normalized entropy,

$$u_h = -\frac{1}{\log |\mathcal{C}_h|} \sum_i \pi_i \log(\pi_i). \quad (4.8)$$

A low uncertainty value indicates that a small number of branches dominate the confidence distribution, allowing more aggressive pruning. In contrast, a high uncertainty value indicates that multiple branches remain plausible, and AFP preserves additional candidates to avoid removing potentially useful reasoning paths.

For example, in Figure 1.1, if the LeBron James frontier produces one dominant branch corresponding to the rookie-season relation, AFP can confidently reduce the frontier. If several branches have similar confidence values, AFP delays aggressive pruning and maintains more alternatives.

4.3.4 Adaptive Frontier Selection

Based on branch confidence and frontier uncertainty, AFP dynamically selects the candidates propagated to the next reasoning hop which is shown in 4.2. Unlike fixed Top- B selection, which retains the same number of candidates at every decision hop, AFP adjusts the retained frontier according to the uncertainty of the current candidate distribution.

Let $n_h = |\mathcal{C}_h|$, and let the candidate probabilities be sorted as $\pi_{(1)} \geq \pi_{(2)} \geq \dots \geq$

$\pi_{(n_h)}$. AFP determines the smallest budget B_h whose cumulative probability mass reaches an adaptive retention threshold γ_h ,

$$B_h = \min \left\{ b \in \{1, \dots, n_h\} \mid \sum_{j=1}^b \pi_{(j)} \geq \gamma_h \right\}. \quad (4.9)$$

Here, γ_h specifies the amount of cumulative branch probability that must be preserved. Rather than using a fixed threshold, AFP adjusts this retained mass according to the normalized frontier uncertainty u_h ,

$$\gamma_h = \gamma_{\min} + u_h(1 - \gamma_{\min}), \quad 0 < \gamma_{\min} < 1. \quad (4.10)$$

As uncertainty decreases ($u_h \rightarrow 0$), γ_h approaches $\gamma_{\min}$, requiring less cumulative probability mass and therefore allowing stronger pruning. Conversely, as uncertainty increases ($u_h \rightarrow 1$), γ_h approaches 1, requiring more probability mass to be preserved and consequently retaining more candidate branches. To avoid arbitrary removal of equally scored branches, AFP applies a tie-aware cutoff. After determining B_h , all candidates whose probability is at least that of the candidate at the cutoff are retained,

$$\mathcal{P}_{h+1}^{\text{AFP}} = \{b_i \in \mathcal{C}_h \mid \pi_i \geq \pi_{(B_h)}\}. \quad (4.11)$$

Thus, candidates tied at the selection boundary are retained together; when all candidate probabilities are identical, the complete candidate set is preserved.

In the Figure 1.1 example, a concentrated candidate distribution allows AFP to retain only the most strongly supported branches, whereas a more uncertain distribution preserves additional alternatives. The selected frontier $\mathcal{P}_{h+1}^{\text{AFP}}$ is then passed to the next graph expansion step, reducing future traversal cost while maintaining the original RoG relation constraints.

4.4 Conservative Final-Hop Strategy

Although AFP effectively reduces unnecessary expansion in intermediate reasoning steps, applying aggressive pruning at the final hop introduces a different risk. At this stage, candidate entities directly correspond to potential answers, and removing a low-scored branch may eliminate the correct answer before answer selection. Therefore, AdaPruner-KGQA adopts a conservative final-hop strategy that separates intermediate search optimization from final answer preservation.

Let $\mathcal{C}_{L-1}$ denote the candidate set generated at the final reasoning hop. Unlike

intermediate frontiers, where AFP selects a reduced subset for further expansion, the final-hop candidates are retained without aggressive pruning, $\mathcal{P}_L^{\text{AFP}} = \mathcal{C}_{L-1}$. The final answer is then determined from the preserved candidate set using the downstream answer selection process. This design ensures that AFP reduces computational overhead by controlling intermediate frontier growth while maintaining the completeness of the final answer space.

For the Figure 1.1 example question, AFP can safely prune irrelevant intermediate branches while the reasoning process is still exploring possible paths. However, once the final hop reaches candidate answer entities representing possible years, all remaining candidates are preserved to avoid removing the correct answer due to an imperfect pruning decision. Therefore, the conservative final-hop strategy provides a balance between search efficiency and answer reliability: AFP aggressively controls intermediate exploration while ensuring that final answer candidates remain available for selection.

Algorithm 1 presents the complete procedure of AdaPruner-KGQA. It first initializes the retrieved reasoning-path set and iterates over each RoG-generated relation plan and its associated topic entities (*Lines 1–4*). For each reasoning hop, the active path prefixes are expanded through the next relation specified by the current relation plan to construct the relation-valid candidate set $\mathcal{C}_h$. If no valid candidate is generated, traversal of the current relation plan is terminated (*Lines 5–8*). At the final hop, all relation-valid candidates are retained without pruning to preserve potential answer entities, while a single intermediate candidate is directly propagated without invoking AFP (*Lines 9–12*). When multiple intermediate candidates exist, AFP is applied to adaptively reduce the frontier (*Lines 13–21*). For each candidate branch, AFP constructs the corresponding semantic, path-context, structural, and reasoning-progress feature representation and computes its relevance logit using the branch scorer (*Lines 14–16*). The logits are normalized into a temperature-scaled probability distribution, from which the frontier uncertainty u_h and adaptive retained-mass threshold γ_h are computed (*Lines 17–19*). Candidates are ranked according to their normalized probabilities, and the smallest budget B_h satisfying the required cumulative probability mass is selected. The top-ranked B_h branches, together with any branches tied at the selection cutoff, are retained as the next reasoning frontier (*Lines 20–21*). These retained branches are then used for subsequent relation expansion (*Lines 22–23*). After completing the current relation plan, the resulting reasoning paths are added to the global retrieval set (*Line 24*). Finally, all retrieved paths across topic entities and relation plans are returned to the unchanged RoG LLM reasoner (*Line 25*).

Algorithm 1: AdaPruner-KGQA Relation-Constrained Retrieval with Adaptive Frontier Pruning

Input : Question q ; topic entities E_q ; relation-plan set $\mathcal{Z}_q$; knowledge graph G ; AFP learnable parameters θ ; temperature T ; minimum retained mass $\gamma_{\min}$

Output : Retrieved reasoning-path set $\mathcal{P}_q^{\text{ret}}$

```

1   $\mathcal{P}_q^{\text{ret}} \leftarrow \emptyset$ ;
2  foreach relation plan  $z = (r_1, \dots, r_L) \in \mathcal{Z}_q$  do
3      foreach topic entity  $e_0 \in E_q$  do
4          Initialize the active path-prefix set as  $\mathcal{P}_0 = \{(e_0)\}$  and the retrieved path set for
            the current plan as empty;
5          for  $h \leftarrow 0$  to  $L - 1$  do
6              Generate the relation-valid candidate set  $\mathcal{C}_h$  by extending the active path
                prefixes through the next planned relation  $r_{h+1}$  in  $G$ ;
7              if  $\mathcal{C}_h = \emptyset$  then
8                  | terminate traversal of the current plan;
9              if  $h = L - 1$  then
10                 | Retain all candidates in  $\mathcal{C}_h$  as complete reasoning paths;
11                 | ;  $\triangleright$  final-hop protection
12             else if  $|\mathcal{C}_h| \leq 1$  then
13                 | Retain the available candidate as the next frontier;
14             else
15                 foreach candidate branch  $b_i \in \mathcal{C}_h$  do
16                     | Construct its semantic, path-context, structural, and
17                     | reasoning-progress feature vector  $\mathbf{x}_i$ ;
18                     | Compute the branch relevance logit,  $\ell_i \leftarrow f_\theta(\mathbf{x}_i)$ ;
19                 Normalize branch logits using temperature-scaled softmax,
20                  $\pi_i \leftarrow \frac{\exp(\ell_i/T)}{\sum_j \exp(\ell_j/T)}$ ;
21                 Compute the normalized uncertainty  $u_h$  of the candidate distribution;
22                  $\gamma_h \leftarrow \gamma_{\min} + u_h(1 - \gamma_{\min})$ ;
23                 Rank candidates according to  $\pi_i$  and determine the smallest budget  $B_h$ 
                whose cumulative probability mass is at least  $\gamma_h$ ;
24                 Retain the highest-ranked  $B_h$  branches, including all branches tied at
                the selection cutoff, as  $\mathcal{P}_{h+1}^{\text{AFP}}$ ;
25             if  $h < L - 1$  then
26                 | Use the retained branches as the next active frontier;
27                 | ;  $\triangleright$  next-hop expansion
28             Add the complete reasoning paths obtained for the current topic entity and
            relation plan to  $\mathcal{P}_q^{\text{ret}}$ ;
29 return  $\mathcal{P}_q^{\text{ret}}$ ;

```

4.5 Training Supervision and Objective

AFP is trained to distinguish between intermediate reasoning branches that can still lead to the correct answer and branches that become irrelevant after a given reasoning step. Unlike the final answer predictor, AFP does not directly learn to predict answers; instead, it learns an answer-preserving branch selection policy for intermediate frontier reduction.

Training instances are constructed exclusively from the training split. Since AFP does not prune the final answer-producing hop, branch-level supervision is required only for intermediate hops $h < L - 1$. For a candidate branch $b_i \in \mathcal{C}_h$, the supervision signal is determined by whether the branch can reach at least one gold answer by following the remaining suffix of the same relation plan. After generating b_i through relation r_{h+1} , the remaining relation suffix is defined as-

$$z_{>h+1} = (r_{h+2}, \dots, r_L). \quad (4.12)$$

The branch label is assigned as,

$$y_i = \begin{cases} 1, & \text{if } b_i \text{ has a valid continuation through } z_{>h+1} \\ & \text{that reaches at least one gold answer } a \in A_q, \\ 0, & \text{otherwise.} \end{cases} \quad (4.13)$$

Therefore, a positive branch is not simply a relation-valid candidate; it must preserve a possible reasoning trajectory toward the answer. Negative branches are particularly informative because they satisfy the same RoG relation constraint as positive candidates but fail to provide an answer-supporting continuation.

Considering the Figure 1.1 example question, multiple intermediate branches may remain valid after RoG [1] generates the relation-constrained candidates. In Figure 4.3, we can see that a branch that continues through the player’s rookie season relation and eventually reaches the answer year receives a positive green label ($y_i = 1$). In contrast, branches describing other valid information about LeBron James, such as team membership or player attributes, are labeled in red and given negative ($y_i = 0$) because they cannot reach the required answer under the remaining relation sequence. This is called binary branch classification. The supervision teaches AFP to remove unproductive expansions while preserving answer-supporting reasoning paths. Plan-hop groups where no candidate can reach a training gold answer are treated as infeasible for answer-preserving branch selection. These cases are excluded from the branch-selection loss rather than being

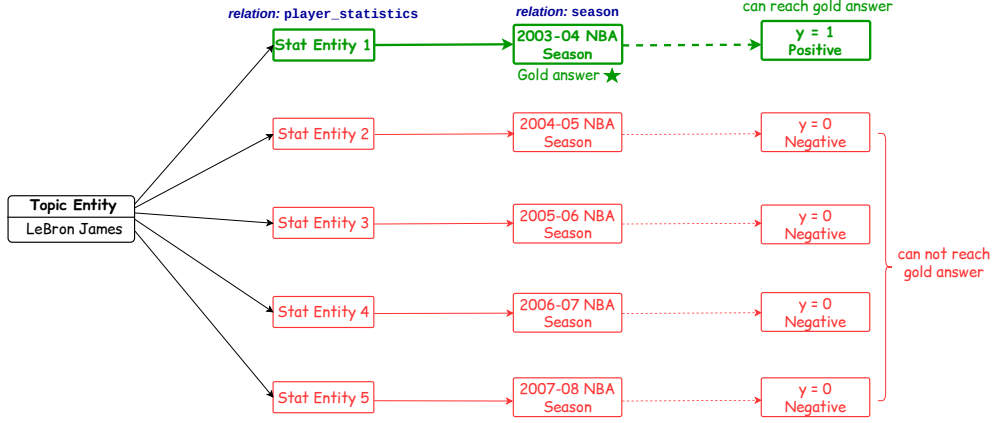


Figure 4.3: Illustration of AFP branch-level supervision. Given the relation plan $z = (\text{player_statistics}, \text{season})$ for the question “What year was LeBron James’ rookie season?”, relation-valid candidate branches are labeled according to whether their remaining relation suffix can reach the gold answer. Only the green branch preserving a gold-answer continuation receive positive supervision ($y_i = 1$), while other valid but unproductive branches in red receive negative labels ($y_i = 0$).

treated as all-negative examples, preventing planner failures or incomplete graph coverage from being incorrectly learned as pruning behavior.

For feasible plan-hop groups, the branch scorer is optimized using binary cross-entropy $\mathcal{L}_{\text{AFP}}$ for binary branch classification over the predicted branch confidence scores s_i :

$$\mathcal{L}_{\text{AFP}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log s_i + (1 - y_i) \log(1 - s_i)], \quad (4.14)$$

where N represents the number of labeled branch instances obtained from feasible training plan-hop groups. All AFP parameters are learned using only the training split. The validation split is used for model selection and calibration of adaptive pruning parameters. The test split is never used for branch labeling, training, hyperparameter selection, calibration, or pruning decisions.

4.6 Search Complexity and Pruning Overhead Analysis

The computational objective of AFP is to reduce graph exploration after an intermediate pruning decision. However, a smaller retained frontier does not automatically imply lower computational cost, since AFP itself requires feature construction, branch scoring, uncertainty estimation, and adaptive selection. We therefore distinguish between *graph-search cost* and *pruning overhead*.

Using the retrieval notation defined in Section 2.4, we quantify graph-search cost by the adjacency entries examined during path expansion. Under RoG traversal, expanding an active path prefix p requires inspecting the adjacency entries of its terminal entity to identify neighbors that satisfy the next planned relation. We therefore define the expansion cost of p as,

$$\delta(p) = |\text{Adj}(\text{last}(p))|, \quad (4.15)$$

where $\delta(p)$ denotes the number of adjacency entries examined while expanding prefix p . This quantity captures the graph-search work incurred before the relation-valid candidate branches are obtained.

Let m denote a retrieval method and $\mathcal{P}_h^m$ its active path-prefix set at hop h . We define E_m as the total *examined-edge cost* of method m , i.e., the total number of adjacency entries inspected during execution of the relation plan,

$$E_m = \sum_{h=0}^{L-1} \sum_{p \in \mathcal{P}_h^m} \delta(p). \quad (4.16)$$

Accordingly, E_{RoG} denotes the examined-edge cost of unpruned RoG, while E_{AFP} denotes the corresponding cost after AFP controls the intermediate frontiers. At question level, these costs are summed over all executed topic entities and RoG-generated relation plans. The relative reduction in graph exploration is measured by Search-Space Reduction (SSR)-

$$\text{SSR} = 1 - \frac{E_{\text{AFP}}}{E_{\text{RoG}}} \quad (4.17)$$

where a larger SSR indicates that AdaPruner-KGQA examines fewer graph adjacency entries than unpruned RoG.

A key property of AFP is that pruning occurs *after* the current relation-valid candidate set $\mathcal{C}_h$ has already been generated. Therefore, AFP cannot recover the

graph-expansion cost already incurred at hop h . Its saving begins at the following hop: candidates removed from $\mathcal{C}_h$ are not admitted to $\mathcal{P}_{h+1}^{\text{AFP}}$ and consequently neither those branches nor their descendants are expanded later. For a pruning decision made at hop h , the downstream examined-edge saving can therefore be expressed as,

$$\Delta E_h^{\text{future}} = \sum_{t=h+1}^{L-1} \left[\sum_{p \in \mathcal{P}_t^{\text{RoG}}} \delta(p) - \sum_{p \in \mathcal{P}_t^{\text{AFP}}} \delta(p) \right] \quad (4.18)$$

Across the complete traversal, the structural graph-search saving is,

$$\Delta E_{\text{graph}} = E_{\text{RoG}} - E_{\text{AFP}}. \quad (4.19)$$

AFP introduces additional computation whenever multiple candidates are available at an intermediate hop. Let, $n_h = |\mathcal{C}_h|$ denote the number of relation-valid candidate branches at such a hop. $d = 27$ denotes the fixed branch-feature dimension and H the hidden dimension of the lightweight feed-forward scorer. The pruning overhead at hop h is decomposed as,

$$C_{\text{AFP}}^{(h)} = C_{\text{feat}}^{(h)} + C_{\text{score}}^{(h)} + C_{\text{unc}}^{(h)} + C_{\text{select}}^{(h)}, \quad (4.20)$$

where $C_{\text{feat}}^{(h)}$ is the cost of constructing the 27-dimensional branch features, $C_{\text{score}}^{(h)}$ is the cost of evaluating the branch scorer, $C_{\text{unc}}^{(h)}$ is the cost of temperature-scaled normalization and entropy computation, and $C_{\text{select}}^{(h)}$ is the cost of ranking and cumulative-mass selection. With the semantic representations cached as in our implementation, feature construction is linear in the candidate-set size, $\mathcal{O}(n_h d)$. Evaluating the two-layer scorer over all candidates requires $\mathcal{O}(n_h d H)$, while temperature-scaled softmax and normalized-entropy computation are $\mathcal{O}(n_h)$. AFP then ranks the candidate probabilities to identify the smallest retained budget satisfying the adaptive cumulative-mass rule, which requires $\mathcal{O}(n_h \log n_h)$ time in the worst case. Therefore, the overall pruning complexity at one AFP decision hop is

$$C_{\text{AFP}}^{(h)} = \mathcal{O}(n_h d H + n_h \log n_h). \quad (4.21)$$

Since $d = 27$ and H are fixed model dimensions, the candidate-ranking term dominates asymptotically, giving a worst-case per-hop pruning complexity of $\mathcal{O}(n_h \log n_h)$ and linear temporary memory in n_h .

Let $\mathcal{H}_{\text{AFP}}$ denote the set of intermediate hops at which AFP is actually invoked;

singleton candidate sets and the protected final hop bypass adaptive pruning. The total pruning overhead for a traversal is then

$$C_{\text{pruner}} = \sum_{h \in \mathcal{H}_{\text{AFP}}} C_{\text{AFP}}^{(h)}. \quad (4.22)$$

Hence, AFP provides a net computational advantage when the computation avoided through future graph expansion exceeds the additional pruning overhead:

$$C_{\text{saved}}^{\text{future}} > C_{\text{pruner}}. \quad (4.23)$$

This distinction is important when interpreting efficiency. A reduction in E_{AFP} provides direct evidence of reduced structural graph exploration, but it does not by itself establish a wall-clock speedup. Accordingly, our evaluation treats examined edges as the primary search-cost measure and reports runtime-related quantities separately so that the computational overhead of AFP is not conflated with structural search-space reduction.

Chapter 5

Experimental Setup

This chapter presents the experimental setup used to evaluate AdaPruner-KGQA, covering the datasets and comparison methods, the controlled RoG reproduction and AFP configuration, search instrumentation and runtime measurement, evaluation metrics, and the staged experimental protocol used to address RQ1–RQ3 while preventing test leakage.

5.1 Datasets

We evaluate AdaPruner-KGQA on the two Freebase-based multi-hop KGQA benchmarks used in the controlled RoG reproduction: WebQuestionsSP (WebQSP) [9] and ComplexWebQuestions (CWQ) [10]. For every direct comparison, the processed questions, linked topic entities, knowledge graph, RoG relation plans, and graph-construction procedure are held fixed. Training data are used for branch-level supervision, the frozen RoG validation splits are used for development and model selection, and the official test sets are reserved for evaluation after the complete AFP configuration is frozen. Dataset statistics used in the controlled experiments are summarized in Table 5.1.

5.1.1 WebQuestionsSP

WebQuestionsSP [9] contains natural-language questions grounded in Freebase and primarily emphasizes short reasoning chains. We use 2,826 training questions, 246 questions in the reproduced RoG validation split, and the complete official test set of 1,628 questions. Although the benchmark predominantly contains short

Dataset	Train	Validation	Test	Max. Frozen RoG Plan Length
WebQSP	2,826	246	1,628	4
CWQ	27,639	3,519	3,531	5

Table 5.1: Dataset splits and maximum relation-plan lengths observed in the frozen RoG artifacts. Plan length refers to planner-generated relations and should not be interpreted as the benchmark’s gold reasoning depth.

reasoning requirements, the frozen RoG planner can generate relation plans longer than the benchmark’s typical gold reasoning depth; the frozen reproduced artifacts contain plans of up to four relations. Under the conservative final-hop policy described in Section 4.4, one-hop plans remain entirely unpruned, whereas longer plans permit AFP only at intermediate hops.

5.1.2 ComplexWebQuestions

ComplexWebQuestions [10] contains compositionally more complex Freebase questions and provides a deeper retrieval setting than WebQSP. We use 27,639 training questions, 3,519 validation questions, and the complete official test set of 3,531 questions. In the frozen reproduced RoG artifacts, generated relation plans contain up to five relations. We therefore distinguish the benchmark’s nominal reasoning depth from the length of relation plans generated by the reproduced planner. The deeper generated plans provide multiple intermediate positions at which AFP may control frontier growth.

5.2 Comparison Methods

This subsection describes the methods used for comparison, including literature-reported KGQA baselines for broader context, the frozen reproduced RoG baseline used for direct evaluation, the controlled pruning baselines used to isolate frontier-selection effects, and recent retrieval-oriented methods included as contextual references.

5.2.1 Literature-Reported KGQA Methods

For broad KGQA context, the overall performance comparison retains the baseline configurations reported in RoG [1], spanning embedding-based, retrieval-based, semantic-parsing, standalone LLM, and LLM with KG-based approaches. These values are reported exactly as literature-reported results and are not presented as methods reproduced in this study.

Published RoG results are likewise distinguished from the RoG implementation reproduced in this work. Consequently, the broad KGQA comparison provides context for answer quality, whereas direct search-efficiency conclusions are drawn only from methods executed under the controlled retrieval setting described below.

5.2.2 Frozen Reproduced RoG Baseline

The primary direct baseline is the reproduced unpruned RoG retrieval pipeline. For every question, the same linked topic entities, frozen RoG-generated relation plans, knowledge graph, graph-construction procedure, relation-matching operation, and retrieval traversal are shared across all controlled comparison methods. Where downstream answer generation is evaluated, the same frozen RoG reasoner and answer-evaluation procedure are also used. The reproduced baseline is frozen before AFP test evaluation and is not retuned using test data.

The reproduced knowledge graph is represented using the same undirected networkx.Graph structure as the controlled RoG implementation. Search instrumentation consists exclusively of read-only counters and does not modify candidate generation, path ordering, path multiplicity, retrieved reasoning paths, or downstream reasoner inputs.

5.2.3 Controlled Pruning Baselines

The direct pruning comparison isolates frontier-selection behavior while holding all other retrieval components fixed. The evaluated configurations are-

1. **RoG (Reproduced):** All relation-valid candidates are retained, reproducing the original relation-constrained retrieval behavior.
2. **Fixed Top- B :** The same trained branch scorer used by AFP ranks the relation-valid candidates, and the highest-scoring B candidates are retained at each intermediate decision hop.

3. **Fixed Threshold:** Candidates whose branch-relevance score exceeds a validation-selected threshold τ are retained, with a non-empty-frontier fallback where required.
4. **Random- B :** A fixed number B of relation-valid candidates are retained uniformly at random at each intermediate decision hop.
5. **Adaptive-Budget Random:** AFP first determines the number of branches that would actually be retained at the current decision hop, including any expansion required by cutoff ties, but the same number of candidates is then selected uniformly at random rather than according to branch relevance.
6. **Adaptive Frontier Pruning (AFP):** the proposed uncertainty-aware policy uses the branch-score distribution to determine an adaptive retained probability mass and retains the corresponding highest-ranked candidates using tie-aware selection.

Fixed Top- B , Fixed Threshold, and AFP use the same candidate-generation procedure and trained branch scorer. Adaptive-Budget Random also uses the AFP score distribution to reproduce AFP’s actual retained count at each encountered decision group, but randomizes candidate identity, thereby separating adaptive budget allocation from relevance-based branch ranking. Random- B instead controls for fixed-budget pruning without learned candidate selection. All pruning methods apply the same conservative final-hop policy.

Hyperparameters are selected on validation data using Search-Space Reduction subject to an Answer Retention constraint of at least 0.99; if no configuration satisfies this condition, Answer Retention is prioritized before Search-Space Reduction. The resulting fixed operating points are $B = 1$ and $\tau = 0.3$ for WebQSP, and $B = 8$ and $\tau = 0.1$ for CWQ. Random- B inherits the corresponding validation-selected B , while Adaptive-Budget Random obtains its local retained count from the frozen AFP policy. Randomized controls use the predeclared seeds {42, 43, 44}.

5.2.4 Recent Contextual Methods

SubgraphRAG [2] and GNN-RAG [12] are included as recent retrieval-oriented KGQA references where compatible published WebQSP or CWQ results are available. Because their retrieval architectures, downstream models, graph processing, and hardware differ from the controlled RoG setting, their published wall-clock

runtimes are not used for direct efficiency claims.

MoKGR [3] is treated as methodological related work on query-adaptive graph pruning rather than as a direct numerical baseline. Its task formulation, datasets, and evaluation protocol differ from the WebQSP/CWQ natural-language KGQA setting considered here.

5.3 Implementation and Controlled Experimental Protocol

This subsection describes the implementation and controlled experimental protocol, including RoG retrieval fidelity, AFP training and hyperparameter selection, the computational environment and frozen AFP configuration, search instrumentation and counter semantics, runtime and downstream reasoner-cost measurement, and the development-oracle and method-freezing procedure used to ensure reproducibility and prevent test leakage.

5.3.1 RoG Retrieval Fidelity

All direct RoG-versus-pruning experiments use relation plans produced by the same reproduced `rmanluo/RoG` planner with up to three predicted relation plans per question. Relation plans are generated once for each split and frozen before the corresponding retrieval experiments; they are never regenerated separately for different pruning configurations. This removes planner variation from the direct comparison and restricts the controlled experimental factor to intermediate frontier selection.

For a question q , each frozen relation plan is executed from its linked topic entity or entities using the relation-matching procedure defined by $\mathcal{M}(e, r)$ in Eq. (2.5). The downstream RoG reasoner is kept unchanged. Thus, AFP modifies neither the relation planner nor the reasoning model; it controls only which relation-valid intermediate candidate branches are propagated to subsequent graph expansion.

Before collecting search statistics, the instrumented retriever is checked against the reproduced RoG retrieval implementation. Equivalence requires identical ordered reasoning paths, including path contents and multiplicity, together with identical question-level answer reachability. The profiler is accepted only after zero retrieval mismatches are observed on the evaluated WebQSP and CWQ questions. This fidelity gate ensures that subsequent differences in controlled retrieval

experiments originate from the frontier-selection policy rather than from instrumentation.

Planner generation additionally uses the verified slow-tokenizer behavior of the reproduced RoG environment. The reproduced planner stack uses transformers==4.44.2, tokenizers==0.19.1, and sentencepiece==0.2.0 packages; relation-token round-trip fidelity is checked before accepting generated planner artifacts.

5.3.2 AFP Training and Hyperparameter Selection

AFP is trained using branch-level supervision derived exclusively from the training split, following Section 4.5. A candidate at an intermediate hop is positive only when it can reach at least one training gold answer by following the exact remaining suffix of the same relation plan. Final-hop candidates are excluded from pruning supervision.

The same suffix-constrained reachability procedure is used for branch-label construction and development-time oracle analysis. Its implementation is validated against direct traversal before large-scale label generation. Plan-hop groups in which no candidate can reach a training gold answer are recorded as infeasible and excluded from the branch-classification loss rather than being treated as ordinary all-negative examples. Consequently, planner or graph-coverage failures are not incorrectly learned as instructions to prune the complete frontier.

The frozen AFP feature representation has $d = 27$ features: 8 semantic, 7 path-context, 8 structural, and 4 reasoning-progress features described in Subsection 4.3.1. Feature normalization statistics are fitted on training data only and are reused unchanged for validation and test inference. Structural features use information already available during the current expansion and do not inspect future candidate neighborhoods solely for pruning.

Semantic features use the frozen pretrained sentence-transformers/all-MiniLM-L6-v2 encoder, based on MiniLM [36] and the Sentence-BERT framework [37]. The encoder contains approximately 22.7 million pretrained parameters and produces 384-dimensional text representations. Readable question, entity, relation, plan, and suffix text is encoded and cached where reusable. Raw unresolved Freebase MIDs are not passed to the sentence encoder as natural-language text; they receive a zero semantic representation together with explicit surface-availability indicators. Knowledge-graph embeddings are not part of the frozen core AFP model.

The branch scorer is a two-layer MLP of the form $27 \rightarrow H \rightarrow 1$ with ReLU ac-

tivation and no dropout. During development, hidden dimensions $\{32, 64, 128\}$, branch BCE versus group-balanced BCE, and seeds $\{42, 43, 44\}$ are evaluated using validation data only. The final selected checkpoints use ordinary branch BCE with $H = 32$ for WebQSP and $H = 64$ for CWQ; both selected checkpoints use seed 42.

Scorer training uses AdamW for 80 full-batch epochs with learning rate 10^{-3} , weight decay 10^{-4} , and gradient clipping at 5.0. No class weighting, dropout, learning-rate scheduler, or validation early stopping is applied.

The adaptive selector is tuned using validation data only. The final frozen configuration uses $(T, \gamma_{\min}) = (0.05, 0.10)$ for WebQSP and $(T, \gamma_{\min}) = (0.10, 0.10)$ for CWQ. Candidate logits are normalized using temperature-scaled softmax, and normalized entropy controls the retained probability mass according to the adaptive rule defined in Section 4.3.4. Selection is tie-aware: candidates tied at the cutoff are retained together, and a fully tied candidate distribution is preserved rather than arbitrarily pruned.

All feature definitions, scorer checkpoints, selector settings, final-hop handling, baseline operating points, and random seeds are frozen before test evaluation. Test gold answers are never used for label construction, scorer training, model selection, temperature calibration, threshold selection, or pruning decisions.

5.3.3 Computational Environment and AFP Configuration

Experiments are executed in version-controlled Kaggle notebook environments, with software and hardware metadata recorded in the corresponding reproducibility manifests. GPU acceleration is used where applicable. In particular, the frozen MiniLM semantic encoder is executed on CUDA for semantic feature construction, whereas the lightweight AFP branch-scoring MLP (Multi-Layer Perceptron) is evaluated on the CPU. Graph traversal is implemented with NetworkX, and scorer training and inference are implemented with PyTorch. Direct wall-clock comparisons are made only between runs executed under matched timing boundaries and compatible hardware conditions; structural search metrics are otherwise reported independently of hardware.

The reproduced RoG knowledge graph uses the same undirected `networkx`. Graph representation across all controlled methods. Planner outputs, frozen feature files, scorer checkpoints, selector configurations, and major experimental outputs are hash-tracked so that artifacts used at later stages can be verified against those selected during development. This preserves the controlled-comparison setting in

Table 5.2: Frozen implementation configuration of the AFP module used in the controlled evaluation

Component	Configuration
Semantic encoder	sentence-transformers/all-MiniLM-L6-v2
Encoder parameters	$\sim 22.7\text{M}$ (pretrained and frozen)
Transformer layers	6
Embedding dimension	384
AFP feature dimension	27
AFP scorer	Linear-ReLU-Linear MLP
Hidden dimension	32 (WebQSP), 64 (CWQ)
Trainable scorer parameters	929 (WebQSP), 1,857 (CWQ)
Scorer objective	Branch-level binary cross-entropy (BCE)
Temperature T	0.05 (WebQSP), 0.10 (CWQ)
Minimum retained probability mass $\gamma_{\min}$	0.10 (both datasets)
Final-hop strategy	Pruning bypassed at the final relation
Tie handling	Tie-aware selection with cutoff expansion

which the questions, topic entities, knowledge graph, planner outputs, and relation plans remain unchanged, while only the intermediate frontier-selection policy differs across retrieval methods. Frozen implementation configuration of the AFP module is depicted in Table 5.2.

The AFP configuration used for the controlled evaluation is frozen before accessing the test split. The selected hidden dimensions are 32 for WebQSP and 64 for CWQ, with branch-level BCE as the scorer objective. The adaptive selector uses $T = 0.05$ for WebQSP and $T = 0.10$ for CWQ, while $\gamma_{\min} = 0.10$ for both datasets. No pruning is applied at the final relation step, and singleton candidate sets bypass pruning. All training, model selection, feature specification, calibration, and selector configuration decisions are completed using the training and validation splits before final test evaluation.

5.3.4 Search Instrumentation and Counter Semantics

Search statistics are recorded at question, relation-plan, and hop level using read-only instrumentation. The counters distinguish path-level execution from unique graph entities and are reported only at aggregation levels for which the corresponding quantity is directly recorded. Because each graph node corresponds to

a KG entity in the reproduced RoG graph, the terms *node* and *entity* refer to the same graph object; raw path or branch counts are not interpreted as unique-entity counts.

Active Path Prefixes Using the notation defined in Section 2.4, $|\mathcal{P}_h|$ denotes the number of active reasoning prefixes available for expansion at hop h . Distinct prefixes are counted separately even when they terminate at the same entity.

Unique Expanded Nodes Unique expanded nodes are counted as distinct KG entities that are actually expanded during retrieval. At the question level, this corresponds to the union of terminal entities of active path prefixes over all executed relation plans and expansion hops. An entity expanded multiple times through different reasoning prefixes or different relation plans is counted once in the question-level unique-expanded-entity total. This quantity is therefore distinct from both the path-prefix count and a single hop’s unique frontier size.

Edges Examined An examined edge is an adjacency entry inspected while an active path prefix is expanded, before determining whether it satisfies the next planned relation. Using the prefix expansion cost $\delta(p)$ defined in Section 4.6, the hop-level examined-edge cost is

$$E_h = \sum_{p \in \mathcal{P}_h} \delta(p). \quad (5.1)$$

Examined edges are used as the primary structural graph-search cost when computing Search-Space Reduction.

Candidate Branches The candidate-branch count at hop h is $|\mathcal{C}_h|$, where $\mathcal{C}_h$ is the relation-valid candidate set defined in Section 2.4. Candidate branches are path extensions rather than unique entities; therefore, distinct prefixes reaching the same endpoint are counted as separate candidate branches.

Unique Frontier Nodes At hop h , the unique frontier set $\mathcal{U}_h = \{last(p) : p \in \mathcal{P}_h\}$ contains the distinct entities represented by the active reasoning prefixes. Accordingly, $|\mathcal{U}_h|$ measures the hop-level unique frontier size. After branch selection, $|\mathcal{U}_{h+1}|$ measures the distinct entities represented in the retained next frontier. This quantity is reported separately from $|\mathcal{P}_h|$ and $|\mathcal{P}_{h+1}|$ because several distinct path prefixes may terminate at the same entity.

Retrieved Reasoning Paths A retrieved reasoning path is a complete path produced after execution of all relations in a selected relation plan. Where this quantity is persisted by the corresponding experiment, the reported count includes all complete reasoning paths obtained across the executed relation plans for a question.

Peak Frontier Peak frontier size measures the largest number of simultaneously active path prefixes encountered during retrieval and is defined as

$$F_q^{\max} = \max_{z,h} |\mathcal{P}_{q,z,h}|. \quad (5.2)$$

This quantity captures the maximum intermediate search pressure observed for question q across its executed relation plans and reasoning hops. Where only a run-level peak is retained, the reported quantity is the maximum of $F_q^{\max}$ over the evaluated questions rather than a question-level mean.

AFP Eligibility and Decision Opportunity A relation plan is considered *AFP-eligible* when $L \geq 2$, because at least one intermediate position exists before the protected final hop. An eligible plan contains an actual *AFP decision opportunity* only when $\exists h < L - 1$ such that $|\mathcal{C}_h| > 1$. Question-level eligibility and decision opportunity are computed using an any-plan criterion.

Downstream Expansion Opportunity As established in Section 4.6, AFP operates only after the current candidate set $\mathcal{C}_h$ has been generated; therefore, current-hop candidate-generation cost cannot be recovered by the pruning decision. For an unpruned relation plan, the subsequent graph work exposed to a decision at hop h is summarized as

$$D_h = \sum_{t=h+1}^{L-1} E_t. \quad (5.3)$$

The quantity D_h measures potential downstream expansion work affected by an earlier pruning decision and is not interpreted as answer-preserving removable work; the latter is examined separately through oracle analysis.

Duplicate Search Pressure Duplicate search pressure is characterized using path-level counts and unique endpoint counts. For non-empty active frontiers, $|\mathcal{P}_h|/|\mathcal{U}_h|$ measures repeated active-prefix endpoints. For non-empty candidate

sets, the candidate-branch count $|\mathcal{C}_h|$ divided by the number of distinct candidate endpoints measures repeated relation-valid extensions terminating at the same entity. These ratios distinguish path-level duplication from genuine growth in the number of unique graph entities represented during retrieval.

5.3.5 Runtime and Reasoner-Cost Measurement

Graph-retrieval latency is measured separately from planner generation, dataset and model loading, semantic-cache construction, post-hoc gold-reachability evaluation, and downstream LLM reasoning. The same frozen relation plans are shared by all directly compared retrieval methods, so planner generation time is not attributed to AFP or to any pruning baseline. For the matched retrieval-timing experiment, wall-clock time is measured using `time.perf_counter_ns` over the execution of the frozen relation-plan traversal. Timing uses one unrecorded warm-up repetition followed by five measured repetitions, with method order randomized within each repetition. The semantic cache is prefilled before measurement and is required to remain unchanged during all timed runs.

Retrieval latency includes the graph traversal and any method-specific frontier-selection computation actually required by the evaluated policy. For AFP this includes feature construction, branch scoring, temperature-scaled normalization, uncertainty estimation, adaptive tie-aware selection, and the downstream graph expansion induced by the retained frontier. RoG performs no pruning and therefore has zero pruning overhead. Random- B is charged only for its random-selection computation; for timing, unnecessary branch-score computation is bypassed because the Random- B policy does not consume score values. An integrity gate verifies that this timing-specific bypass preserves exactly the same selected branches and structural retrieval outputs as the frozen controlled Random- B run. Fixed Top- B and Fixed Threshold retain their scorer and selection costs, while Adaptive-Budget Random retains the scorer and AFP-budget computation required to reproduce AFP’s local retained count before randomizing candidate identity.

For AFP, the conceptual pruning overhead T_{pruner} is decomposed as

$$T_{\text{pruner}} = T_{\text{feature}} + T_{\text{score}} + T_{\text{unc}} + T_{\text{select}}, \quad (5.4)$$

where T_{unc} includes temperature-scaled normalization and normalized-entropy computation. In the timing implementation, wall-clock instrumentation records the combined feature-and-scoring block and the policy-selection block directly; the latter includes uncertainty computation when required by the policy. Pruning

overhead is measured directly rather than being inferred from the difference between a pruning method’s total latency and RoG latency. One-time model loading and semantic-cache construction are excluded from online retrieval timing.

Because pruning can alter the amount of retrieved evidence supplied to the unchanged RoG reasoner, retrieved-path count, serialized evidence size, reasoner-input tokens, reasoner-output tokens, and reasoner latency are evaluated separately during downstream reasoning experiments where the corresponding artifacts are available. The controlled post-planning pipeline time T_{ctrl} is

$$T_{\text{ctrl}} = T_{\text{retrieval}} + T_{\text{reason}}, \quad (5.5)$$

while planner-inclusive execution time T_{e2e} , where reported, is

$$T_{\text{e2e}} = T_{\text{plan}} + T_{\text{retrieval}} + T_{\text{reason}}. \quad (5.6)$$

A reduction in examined edges is therefore not interpreted as a wall-clock speedup unless the corresponding timing measurement demonstrates such a reduction.

5.3.6 Development Oracle and Method-Freezing Protocol

Search-space characterization used to guide method development is performed on training and validation data before final test evaluation. Validation relation plans are generated and frozen before development profiling, oracle analysis, scorer selection, and selector tuning.

A suffix-constrained oracle diagnostic identifies intermediate candidate branches that can still reach a gold answer through the exact remaining relation-plan suffix. The oracle is used only to quantify development-time answer-preserving pruning headroom and is never available to AFP during inference. The same suffix-reachability implementation used for supervision is independently checked against direct traversal before label generation.

After development profiling, scorer selection, controlled validation comparison, and component ablations, the feature specification, scorer weights, adaptive selector, final-hop strategy, comparison baselines, and random seeds are frozen. Test data are not subsequently used to revise these choices. Any test-set oracle analysis is performed only after the complete method is frozen and is treated strictly as a post-hoc diagnostic.

5.4 Evaluation Metrics

Evaluation considers graph-retrieval behavior, answer reachability, final KGQA effectiveness, and computational efficiency. Final test metrics are computed over the complete official test sets using the frozen experimental configuration.

Hits@1 and F1 Hits@1 and F1 follow the frozen reproduced RoG answer-parsing and scoring protocol. The same downstream reasoner and evaluator are used for unpruned RoG and every controlled pruning configuration for which downstream reasoning results are reported.

Question-Level Retrieval Reachability and Coverage A gold answer is considered reachable for question q when at least one executed relation plan produces a complete retrieved path whose endpoint is a gold answer entity. Let $\mathcal{Q}_{\text{reach}}^m$ denote the set of questions satisfying this condition under method m . Retrieval coverage is defined as

$$\text{Coverage}(m) = \frac{|\mathcal{Q}_{\text{reach}}^m|}{|\mathcal{Q}_{\text{eval}}|}. \quad (5.7)$$

The reachable set of unpruned RoG is computed once on the corresponding evaluation split and then held fixed for Answer Retention comparisons.

Answer Retention Answer Retention (AR) measures whether pruning preserves questions whose gold answers are already reachable under unpruned RoG:

$$\text{AR}(m) = \frac{|\mathcal{Q}_{\text{reach}}^m \cap \mathcal{Q}_{\text{reach}}^{\text{RoG}}|}{|\mathcal{Q}_{\text{reach}}^{\text{RoG}}|}. \quad (5.8)$$

Questions already unreachable under unpruned RoG are therefore not counted as pruning failures. AR is reported separately from Hits@1 and F1 because preserving answer reachability does not guarantee that the downstream reasoner produces the same final answer from a modified evidence set.

Search-Space Reduction Examined edges are used as the primary measure of structural graph-search cost. Search-Space Reduction (SSR) is computed according to Eq. (4.17), using the total examined-edge counts of the pruning method and the corresponding unpruned RoG traversal over the same questions. Supporting counters such as active prefixes, candidate branches, unique entities, frontier

size, and retrieved paths are interpreted separately and are reported only when directly persisted at the corresponding aggregation level; none is substituted for the examined-edge definition of SSR.

Dataset-level SSR is the primary structural search-reduction measure. Supplementary analyses may additionally report SSR over baseline-defined AFP-eligible and decision-opportunity subsets; these subsets are determined from the unpruned traversal before applying any pruning configuration.

Reasoner-Input and End-to-End Efficiency The downstream computational effect of pruning is evaluated separately from structural graph reduction. Where the corresponding downstream artifacts are available, retrieved reasoning paths, serialized evidence size, reasoner-input tokens, reasoner-output tokens, reasoner latency, controlled post-planning latency T_{ctrl} , and planner-inclusive latency T_{e2e} are reported independently of SSR.

Search-Profile Statistics Supporting search-profile statistics include active path prefixes, unique expanded entities, examined edges, candidate branches, unique frontier entities, retrieved reasoning paths, peak frontier size, AFP eligibility, decision-opportunity prevalence, downstream expansion opportunity, and duplicate search pressure. These quantities are reported at the question, plan, hop, or run level according to how they are directly recorded by the corresponding experiment. Because KG search distributions are strongly skewed, distributions are summarized using medians and upper-tail percentiles in addition to means and maxima where appropriate.

Statistical Reporting Random- B and Adaptive-Budget Random are evaluated using the predeclared seeds $\{42, 43, 44\}$. The final AFP scorer checkpoint and selector configuration are fixed by validation-stage selection before test evaluation and are not selected according to test performance. Final method comparisons use the same questions across methods, enabling paired question-level analyses and confidence intervals where reported. Timing results are summarized over the matched repeated measurements rather than from a single wall-clock execution.

5.5 Research Questions and Experimental Protocol

The experiments are organized around the three research questions defined in Section 3. For each RQ, the experimental setup specifies how the question is evaluated, followed by a concise summary of the corresponding findings.

RQ1: *Where and to what extent does search-space growth occur during RoG’s relation-constrained graph retrieval?*

Setup: Unpruned RoG is executed with the read-only instrumentation defined in Section 5.3.4. The analysis examines active path prefixes, unique expanded entities, examined edges, candidate branches, unique frontier entities, retrieved paths, peak frontier size, branching prevalence, duplicate search pressure, AFP eligibility, decision-opportunity prevalence, and downstream expansion opportunity across reasoning hops and datasets, using each counter only at an aggregation level directly supported by the profiling artifacts.

Findings: Relation-constrained retrieval substantially limits exploration but does not eliminate entity-level branching. Search cost is strongly skewed, with a small number of high-branching cases producing large candidate frontiers and examined-edge counts. The extreme tail is substantially larger on CWQ, where up to 15,624 candidate branches and 43,473 examined edges are observed.

RQ2: *Can adaptive frontier pruning reduce unnecessary graph exploration more effectively than unpruned RoG and simple fixed-pruning strategies?*

Setup: AFP is compared directly with unpruned RoG, Fixed Top-*B*, Fixed Threshold, Random-*B*, and Adaptive-Budget Random under the same frozen relation plans and graph traversal. SSR and supporting search counters quantify structural search reduction. Timing and pruning-overhead measurements are examined where the corresponding controlled artifacts are available. Random-*B* and Adaptive-Budget Random provide diagnostic controls for separating frontier-size reduction, adaptive budget allocation, and relevance-based branch selection. Feature-family and selector ablations isolate the contribution of AFP components.

Findings: AFP reduces examined-edge search cost relative to unpruned RoG by 2.24% on WebQSP and 5.05% on CWQ, while also reducing candidate branches and active prefixes. However, Random-*B*, Fixed Top-*B*,

and, on CWQ, Adaptive-Budget Random achieve larger SSR, showing that AFP is not the most aggressive pruning strategy. The ablation results identify uncertainty adaptation and temperature scaling as the components with the clearest influence on the efficiency–retention trade-off.

RQ3: *Can the reduced search space preserve RoG-reachable answers and final KGQA performance while improving retrieval efficiency?*

Setup: Answer Retention, retrieval coverage, Hits@1, and F1 are evaluated using the frozen retrieval configurations and the unchanged RoG downstream reasoner. Retrieved-path count, reasoner-input size, reasoning latency, and controlled execution time are examined where the corresponding downstream artifacts are available. These outcomes are interpreted jointly with the structural evidence established under RQ2. Answer reachability and final QA quality are treated separately so that downstream reasoning errors are not attributed to retrieval pruning.

Findings: AFP retains 99.20% of RoG-reachable answers on WebQSP and 98.72% on CWQ. Final QA performance also remains close to the reproduced RoG baseline: WebQSP changes from 86.1% to 85.7% Hits@1 and from 70.1% to 69.9% F1, while CWQ changes from 60.9% to 60.4% Hits@1 and from 53.7% to 53.2% F1. AFP also reduces retrieved-path count by 21.8% on WebQSP and 27.0% on CWQ, with corresponding reasoner-input token reductions of 10.6% and 6.2%.

The experimental sequence is staged to prevent test leakage. The profiler is first validated for retrieval fidelity, after which validation relation plans are frozen for development profiling and oracle analysis. Full training relation plans are then frozen, branch supervision is constructed from training gold answers only, and the scorer is trained. Scorer selection, pruning-baseline selection, adaptive-selector tuning, and component-ablation choices use training and validation data only. The complete AFP configuration and evaluation policies are frozen before final test retrieval and downstream reasoning. Any final-test ablations use only predeclared configurations and are not retuned from test outcomes.

The test sets are not used to train the scorer, construct branch labels, select the hidden dimension or loss, tune B or τ , calibrate T or $\gamma_{\min}$, select a random seed, modify the feature representation, or alter the reproduced RoG retrieval procedure. Any test-set oracle analysis is performed only after the method and evaluation configuration are frozen and is reported solely as post-hoc diagnostic analysis.

Chapter 6

Results and Discussion

This chapter presents the experimental results under the protocol described in previous chapter, covering the fidelity of the reproduced RoG baseline, the search-space characterization of unpruned RoG (RQ1), the search-space reduction achieved through Adaptive Frontier Pruning relative to unpruned RoG and fixed-pruning baselines (RQ2), and the resulting Answer Retention and KGQA performance (RQ3). The chapter further examines the efficiency–effectiveness trade-off across pruning strategies, reports an ablation study isolating the contribution of AFP’s individual components, and presents a failure analysis distinguishing planner, coverage, pruning, and reasoning errors. The results are situated against recent graph-retrieval methods before closing with the limitations and threats to validity affecting their interpretation.

6.1 Baseline Reproduction and Overall KGQA Comparison

We first assess the fidelity of the reproduced RoG implementation before evaluating adaptive frontier pruning. Table 6.1 presents the literature-reported methods included in the main RoG comparison, the published RoG results, our reproduced RoG results, and AdaPruner-KGQA. Published results are included only as broader KGQA context and are clearly distinguished from the controlled experiments conducted in this study.

On WebQSP, the reproduced RoG system achieves 86.1% Hits@1 and 70.1% F1 compared with the published RoG values of 85.7% and 70.8%, respectively. The

Category	Method	WebQSP		CWQ	
		Hits@1	F1	Hits@1	F1
Embedding-Based	KV-Mem [14]	46.7	34.5	18.4	15.7
	EmbedKGQA [4]	66.6	–	45.9	–
	NSM [15]	68.7	62.8	47.6	42.4
	TransferNet [34]	71.4	–	48.6	–
	KGT5 [18]	56.1	–	36.5	–
Retrieval-Based	GraftNet [5]	66.4	60.4	36.8	32.7
	PullNet [6]	68.1	–	45.9	–
	SR+NSM [39]	68.9	64.1	50.2	47.1
	SR+NSM+E2E [39]	69.5	64.1	49.3	46.3
Semantic Parsing	SPARQA [27]	–	–	31.6	–
	QGG [26]	73.0	73.8	36.9	37.4
	ArcaneQA [21]	–	75.3	–	–
	RnG-KBQA [38]	–	76.2	–	–
LLM	Flan-T5-xl [23]	31.0	–	14.7	–
	Alpaca-7B [30]	51.8	–	27.4	–
	LLaMA2-Chat-7B [13]	64.4	–	34.6	–
	ChatGPT [16]	66.8	–	39.9	–
	ChatGPT+CoT [17]	75.6	–	48.9	–
LLM and KG	KD-CoT [24]	68.6	52.5	55.7	–
	UniKGQA [25]	77.2	72.2	51.2	49.1
	DECAF [22]	82.1	78.8	–	–
	RoG (Published) [1]	85.7	70.8	62.6	56.2
Controlled	RoG (Reproduced)	86.1	70.1	60.9	53.7
	AdaPruner-KGQA	85.7	69.9	60.4	53.2

Table 6.1: Overall KGQA performance on WebQSP and CWQ. Literature-reported values are taken from the original publications and the main RoG comparison. RoG (Reproduced) denotes our frozen reproduction. Best results are represented in bold format.

Dataset	Test Questions	Gold-Reachable	Coverage (%)
WebQSP	1,628	1,369	84.09
CWQ	3,531	2,422	68.59

Table 6.2: Frozen reproduced RoG retrieval baseline.

reproduced Hits@1 is 0.4 percentage points higher, while F1 is 0.7 points lower. On CWQ, the reproduced implementation achieves 60.9% Hits@1 and 53.7% F1 compared with the published values of 62.6% and 56.2%, corresponding to differences of 1.7 and 2.5 percentage points, respectively.

The reproduction remains sufficiently close to the published RoG results to provide the controlled baseline used in the subsequent retrieval experiments. Importantly, no test-set tuning was performed to force the reproduced scores toward the published values.

The reproduced RoG retrieval process reaches at least one gold answer for 1,369 of the 1,628 WebQSP questions and 2,422 of the 3,531 CWQ questions, corresponding to coverage of 84.09% and 68.59%, respectively. These sets are frozen and used as the denominators for Answer Retention throughout the pruning experiments.

AdaPruner-KGQA remains close to the reproduced RoG performance. Hits@1 decreases by 0.4 percentage points on WebQSP and 0.5 points on CWQ, while F1 decreases by 0.2 and 0.5 points, respectively. The following experiments examine the corresponding reduction in graph-search cost.

6.2 RQ1: Search-Space Characterization of RoG

RQ1 investigates where and to what extent residual search-space growth remains after RoG has already restricted traversal using predicted relation paths. Profiling is performed at question, relation-plan, and hop level.

Four graph-search quantities are kept conceptually distinct. *Active path prefixes* count the reasoning branches available for expansion. *Unique Expanded Nodes* count distinct graph entities actually expanded. *Candidate Branches* count relation-valid path extensions generated before pruning, while *Unique Frontier Nodes* count distinct entities represented by the resulting frontier. Finally, *Edges Examined* measures the adjacency entries inspected during traversal and is used as

Dataset	Metric	Median	P90	P95	Maximum
WebQSP	Active Path Prefixes	1.000	4.000	10.000	227
	Unique Expanded Nodes	1.000	17.000	29.000	158
	Edges Examined	147.000	1,542.000	1,739.000	5,211
	Candidate Branches	2.000	19.000	31.000	730
	Unique Frontier Nodes	1.000	4.000	10.000	157
CWQ	Active Path Prefixes	1.000	4.000	11.000	396
	Unique Expanded Nodes	3.000	22.000	37.500	199
	Edges Examined	107.000	1,433.000	1,745.000	43,473
	Candidate Branches	1.000	23.000	44.000	15,624
	Unique Frontier Nodes	1.000	4.000	10.000	180

Table 6.3: Search-space characterization of unpruned RoG. Values summarize the observed hop-level distributions.

the primary measure of graph-search computation.

The profiling results show that RoG’s relation guidance substantially restricts relation-level exploration but does not eliminate entity-level branching. Multiple entities can satisfy the same planned relation, producing multiple relation-valid candidate branches.

The distributions are strongly skewed. Although the median hop-level observation has a small search frontier, the P90, P95, and maximum values are considerably larger. A relatively small number of high-branching observations therefore account for much larger search costs.

The extreme tail is more pronounced on CWQ. The P95 values for Candidate Branches and Edges Examined are similar between CWQ (44 branches and 1,745 edges) and WebQSP (31 branches and 1,739 edges), but the maximum values differ substantially. CWQ reaches 15,624 candidate branches and 43,473 examined edges, compared with 730 candidate branches and 5,211 examined edges on WebQSP. This heavier tail is consistent with the greater multi-hop complexity of CWQ.

The difference between Candidate Branches and Unique Frontier Nodes is also important. Multiple reasoning prefixes may terminate at the same graph entity, and therefore raw path-level branch counts cannot be interpreted as unique entity counts. This observation motivates the separate reporting of path-level and unique-node metrics throughout the remainder of the study.

These results answer RQ1 by demonstrating that residual search growth remains after relation-level planning and is concentrated in entity/path branching within relation-valid frontiers. The extreme cases are substantially larger on CWQ, where

Dataset	Method	Active Prefixes	Candidate Branches	Edges Examined	SSR (%)
WebQSP	RoG	9.69	31.42	1,425.59	0.00
	Random- <i>B</i>	3.96	20.53	1,365.54	4.21
	Fixed Top- <i>B</i>	6.88	22.23	1,378.30	3.32
	Fixed Threshold	7.19	29.94	1,408.46	1.20
	Adaptive-Budget-Random	8.18	25.89	1,392.92	2.29
	AFP	8.18	26.01	1,393.67	2.24
CWQ	RoG	15.41	86.06	1,648.01	0.00
	Random- <i>B</i>	9.85	50.02	1,511.71	8.27
	Fixed Top- <i>B</i>	11.80	52.26	1,525.01	7.46
	Fixed Threshold	14.79	84.34	1,641.03	0.42
	Adaptive-Budget-Random	12.75	62.63	1,545.74	6.21
	AFP	12.78	65.24	1,564.86	5.05

Table 6.4: Search-space comparison of RoG and pruning methods. Structural counts are mean values per question. SSR is calculated from total Edges Examined relative to unpruned RoG.

intermediate branches can propagate through longer reasoning plans.

6.3 RQ2: Search-Space Reduction through Adaptive Frontier Pruning

RQ2 evaluates whether AFP can reduce graph exploration relative to unpruned RoG and simple fixed-pruning strategies. All methods use exactly the same questions, topic entities, relation plans, graph representation, relation matching, final-hop protection, and downstream reasoning procedure. The only difference is the intermediate frontier-selection strategy.

Table 6.4 reports the principal structural search metrics. Values represent mean counts per question. AFP reduces graph-search cost relative to unpruned RoG. On WebQSP, mean Edges Examined decreases from 1,425.59 to 1,393.67 per question, corresponding to an SSR of 2.24%. Candidate Branches decrease from 31.42 to 26.01, while mean Active Path Prefixes decrease from 9.69 to 8.18.

On CWQ, mean Edges Examined decreases from 1,648.01 to 1,564.86, giving an SSR of 5.05%. Candidate Branches decrease from 86.06 to 65.24, while mean Active Path Prefixes decrease from 15.41 to 12.78. The reduction is larger than on WebQSP, although Random-*B*, Fixed Top-*B*, and Adaptive-Budget-Random achieve greater SSR on CWQ.

Random-*B* achieves the largest reduction in Edges Examined on both datasets,

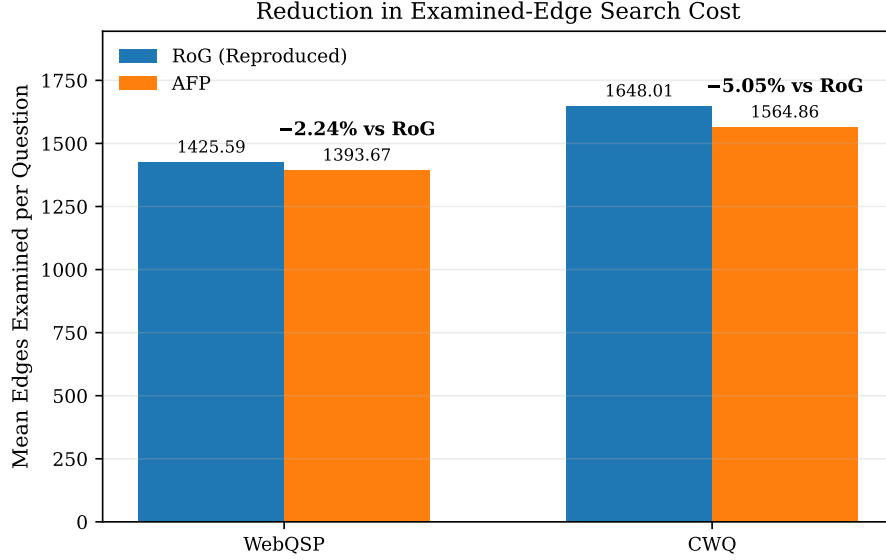


Figure 6.1: Mean Edges Examined for the reproduced RoG baseline and AFP. AFP reduces examined-edge search cost by 2.24% on WebQSP and 5.05% on CWQ.

with 4.21% SSR on WebQSP and 8.27% on CWQ. However, it also produces the lowest Answer Retention on WebQSP, as shown in Section 6.4. Fixed Top- B achieves 3.32% and 7.46% SSR, while Fixed Threshold produces only 1.20% and 0.42% SSR on WebQSP and CWQ, respectively. Adaptive-Budget-Random yields 2.29% SSR on WebQSP and 6.21% on CWQ. Thus, its reduction is close to AFP on WebQSP but larger than AFP on CWQ.

Overall, RQ2 is supported by reductions in Edges Examined, Candidate Branches, and Active Path Prefixes relative to unpruned RoG. AFP lowers mean Edges Examined by 31.92 per WebQSP question and 83.15 per CWQ question, corresponding to 2.24% and 5.05% SSR. The baseline comparison also shows that AFP is not the most aggressive pruning strategy; its effectiveness must therefore be interpreted jointly with Answer Retention.

6.4 RQ3: Answer Retention and KGQA Performance

RQ3 investigates whether the reduced graph search preserves answers that were reachable under the frozen RoG baseline and whether final KGQA effectiveness

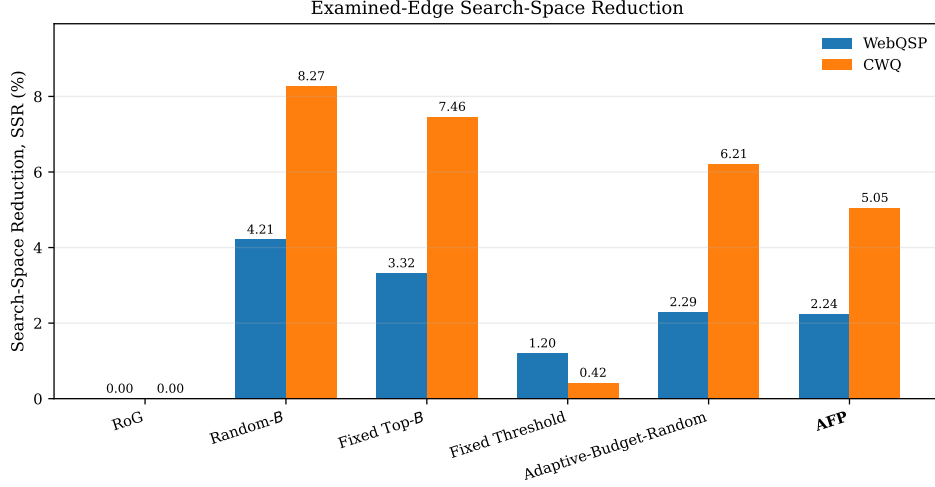


Figure 6.2: Search-Space Reduction across the evaluated pruning strategies. SSR is computed from Edges Examined relative to unpruned RoG.

remains close to the reproduced system. All the results are portrayed in the Table 6.5.

On WebQSP, AFP preserves 99.20% of the questions whose answers are reachable under unpruned RoG. This matches Adaptive-Budget-Random and exceeds Fixed Top-*B* (98.17%), Fixed Threshold (97.95%), and Random-*B* (94.59%).

On CWQ, AFP retains 98.72% of the baseline-reachable questions. This is slightly below Adaptive-Budget-Random (98.76%), Random-*B* (98.93%), Fixed Top-*B* (99.71%), and Fixed Threshold (100.00%). AFP therefore does not provide the highest Answer Retention among the pruning strategies on CWQ.

These Answer Retention values are obtained alongside 2.24% and 5.05% reductions in examined-edge search cost on WebQSP and CWQ, respectively. Final KGQA performance also remains close to the reproduced RoG baseline. AFP achieves 85.7% Hits@1 and 69.9% F1 on WebQSP, compared with 86.1% and 70.1% for reproduced RoG. On CWQ, AFP achieves 60.4% Hits@1 and 53.2% F1, compared with 60.9% and 53.7% for reproduced RoG.

The relationship between search reduction and answer preservation is not monotonic. Random-*B* produces the highest SSR on both datasets but has the lowest Answer Retention on WebQSP. Fixed Threshold preserves all RoG-reachable answers on CWQ but reduces examined-edge cost by only 0.42%. AFP therefore represents an intermediate operating point rather than a uniformly dominant prun-

Dataset	Method	AR (%)	SSR (%)
WebQSP	RoG	100.00	0.00
	Random- <i>B</i>	94.59	4.21
	Fixed Top- <i>B</i>	98.17	3.32
	Fixed Threshold	97.95	1.20
	Adaptive-Budget-Random	99.20	2.29
	AFP	99.20	2.24
CWQ	RoG	100.00	0.00
	Random- <i>B</i>	98.93	8.27
	Fixed Top- <i>B</i>	99.71	7.46
	Fixed Threshold	100.00	0.42
	Adaptive-Budget-Random	98.76	6.21
	AFP	98.72	5.05

Table 6.5: Answer preservation across pruning methods. Answer Retention is computed only over questions reachable by the frozen unpruned RoG baseline.

ing configuration.

6.5 Efficiency–Effectiveness Trade-Off

The efficiency–effectiveness trade-off is analyzed as part of RQ3 rather than as a separate research question. Figures 6.3 and 6.4 compare Search-Space Reduction with Answer Retention for the evaluated pruning strategies.

On WebQSP, AFP and Adaptive-Budget-Random provide the highest Answer Retention among the pruning methods at 99.20%, while achieving 2.24% and 2.29% SSR, respectively. Random-*B* and Fixed Top-*B* reduce more graph search but preserve fewer RoG-reachable answers.

On CWQ, Fixed Threshold preserves 100.00% of RoG-reachable answers but achieves only 0.42% SSR. Fixed Top-*B* reaches 7.46% SSR while maintaining 99.71% AR. AFP achieves 5.05% SSR with 98.72% AR. Thus, no single pruning strategy dominates the efficiency–effectiveness trade-off on both datasets.

6.6 Ablation Study

The ablation study evaluates the contribution of the principal AFP components. Each configuration removes one component from the full method rather than progressively constructing a different architecture. This makes the effect of each component directly interpretable.

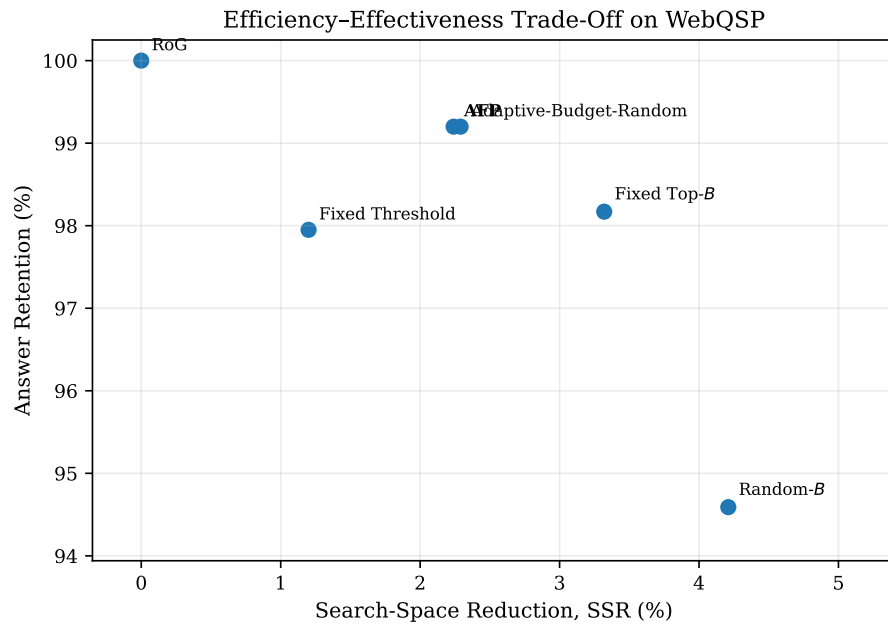


Figure 6.3: Efficiency–effectiveness trade-off on WebQSP. Higher SSR indicates greater examined-edge reduction, while higher Answer Retention indicates stronger preservation of RoG-reachable answers.

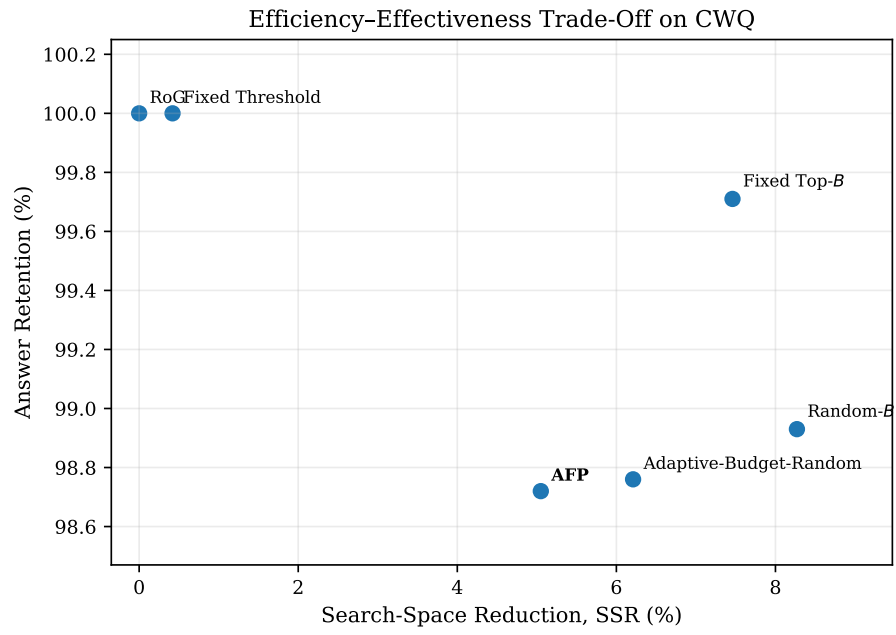


Figure 6.4: Efficiency-effectiveness trade-off on CWQ. Higher SSR indicates greater examined-edge reduction, while higher Answer Retention indicates stronger preservation of RoG-reachable answers.

Dataset	Configuration	Input Dim.	SSR (%)	Δ SSR (pp)	AR (%)	Δ AR (pp)	Edges
WebQSP	Full AFP	27	2.239	–	99.196	–	2,268,901
	w/o Semantic	19	1.431	-0.808	99.489	+0.292	2,287,642
	w/o Path Context	20	2.677	+0.439	99.196	–	2,258,723
	w/o Structural	19	2.501	+0.262	99.123	-0.073	2,262,810
	w/o Progress	23	2.151	-0.087	99.196	–	2,270,931
	w/o Uncertainty	27	3.168	+0.930	98.320	-0.877	2,247,324
	w/o Temperature	27	0.001	-2.237	100.000	+0.804	2,320,825
CWQ	Full AFP	27	5.045	–	98.720	–	5,525,531
	w/o Semantic	19	3.631	-1.415	98.637	-0.083	5,607,861
	w/o Path Context	20	5.490	+0.445	98.803	+0.083	5,499,652
	w/o Structural	19	7.210	+2.164	98.927	+0.206	5,399,588
	w/o Progress	23	6.038	+0.992	98.844	+0.124	5,467,782
	w/o Uncertainty	27	11.881	+6.836	96.243	-2.477	5,127,726
	w/o Temperature	27	0.099	-4.946	100.000	+1.280	5,813,355

Table 6.6: AFP ablation results

The evaluated configurations include Full AFP and the removal of the semantic, path-context, structural/traversal, progress, uncertainty, and temperature components.

The clearest ablation effect comes from removing uncertainty adaptation. On WebQSP, SSR increases from 2.239% to 3.168%, while Answer Retention decreases from 99.196% to 98.320%. On CWQ, SSR increases from 5.045% to 11.881%, while Answer Retention decreases from 98.720% to 96.243%. This is the largest retention loss among the evaluated variants and shows that uncertainty adaptation limits aggressive pruning when the selector is less confident.

Removing temperature has the opposite effect. SSR falls to 0.001% on WebQSP and 0.099% on CWQ, while Answer Retention rises to 100.000% on both datasets. Under the frozen selector, temperature scaling is therefore critical for producing meaningful pruning rather than retaining almost the entire relation-valid frontier.

Removing semantic information decreases SSR on both datasets. On WebQSP, SSR falls to 1.431% while AR rises to 99.489%. On CWQ, both SSR and AR decrease to 3.631% and 98.637%, respectively. The semantic component therefore has a dataset-dependent effect on the efficiency–retention balance.

The path-context ablation produces only moderate changes. It increases SSR to 2.677% on WebQSP with unchanged AR and to 5.490% on CWQ with a 0.083-point increase in AR. Removing structural features increases SSR to 2.501% on WebQSP with a small AR decrease, whereas on CWQ it increases both SSR and AR to 7.210% and 98.927%. These results indicate that the structural component is not uniformly beneficial at the frozen operating point.

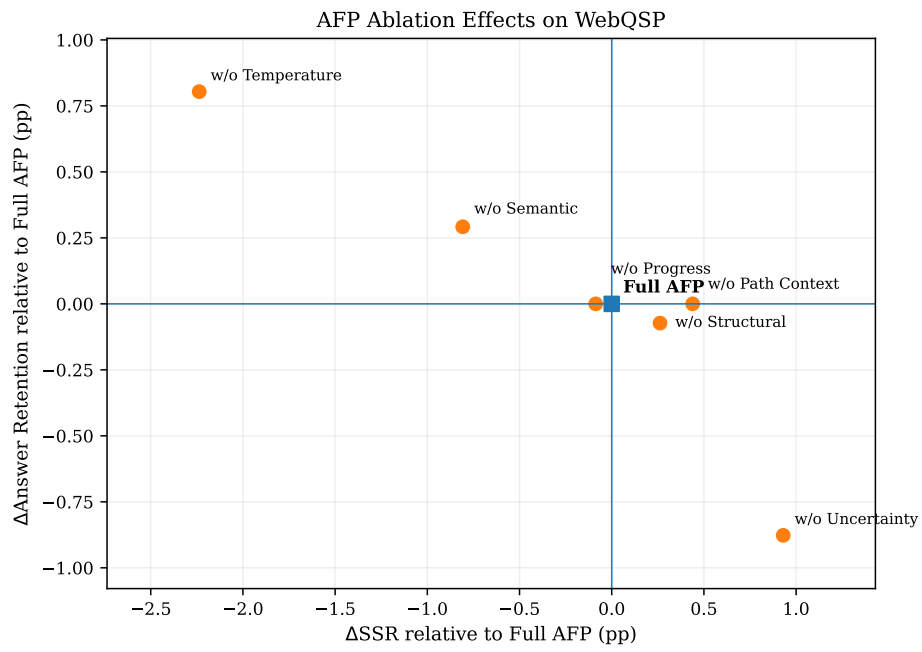


Figure 6.5: Effect of AFP component ablations on WebQSP relative to Full AFP. Positive Δ SSR indicates greater search-space reduction, while positive Δ AR indicates higher Answer Retention.

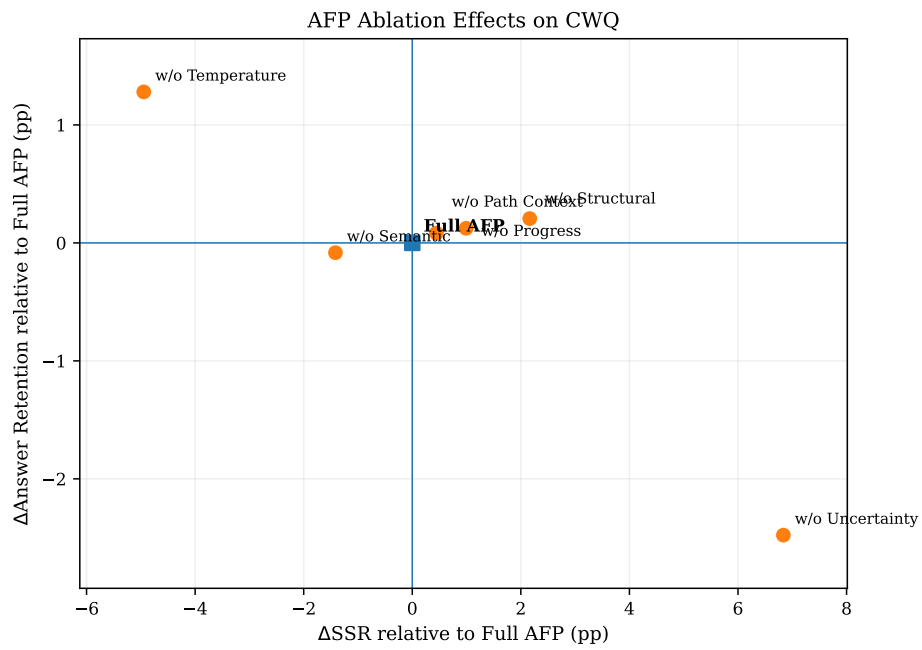


Figure 6.6: Effect of AFP component ablations on CWQ relative to Full AFP. Positive Δ SSR indicates greater search-space reduction, while positive Δ AR indicates higher Answer Retention.

Removing progress produces little change on WebQSP, where SSR becomes 2.151% and AR remains 99.196%. On CWQ, SSR increases to 6.038% and AR to 98.844%. Progress information therefore has a smaller and dataset-dependent contribution compared with uncertainty and temperature.

Overall, the ablation results show that uncertainty adaptation and temperature scaling have the clearest effects on AFP’s pruning behavior. The remaining feature groups produce smaller or dataset-dependent changes, and no individual feature group is uniformly beneficial across both datasets.

6.7 Failure and Case Analysis

Failure analysis separates errors according to the stage at which an answer-supporting path becomes unavailable. Four categories are considered: planner failure, graph/subgraph coverage failure, pruning failure, and downstream reasoning failure.

A *planner failure* occurs when no executed relation plan expresses the relation sequence required for an answer-supporting graph path. A *graph/subgraph coverage failure* occurs when the required path is unavailable in the graph used by retrieval. Neither category is caused by AFP.

A *pruning failure* is defined more strictly: unpruned RoG must reach a gold answer, while AFP removes an intermediate prefix required to preserve that reachability. AFP loses retrieval reachability for 11 of the 1,369 baseline-reachable WebQSP questions and 31 of the 2,422 baseline-reachable CWQ questions. These counts correspond to the 99.20% and 98.72% Answer Retention values, respectively.

A *reasoning failure* occurs when a gold answer remains available in the retrieved reasoning paths but the unchanged RoG LLM reasoner fails to produce the corresponding final answer.

The ablation results help explain the pruning failures. Removing uncertainty substantially increases SSR but also produces the largest reduction in Answer Retention, particularly on CWQ. The adaptive uncertainty mechanism therefore acts as a safeguard against overly aggressive pruning, although it cannot preserve every answer-supporting prefix.

Conversely, removing temperature almost eliminates pruning and restores 100% Answer Retention. Together, these results show that AFP’s observed behavior arises from the interaction between score calibration and uncertainty-adaptive frontier selection rather than from a fixed pruning budget.

6.8 Contextual Comparison with Recent Methods

The controlled experiments above provide the primary evidence for AdaPruner-KGQA because all major retrieval variables except branch selection are held fixed. Recent graph-retrieval systems are therefore discussed as contextual references rather than hardware-controlled efficiency baselines.

SubgraphRAG [2] constructs flexible question-specific subgraphs using a lightweight learned retriever, while GNN-RAG [12] employs graph-neural retrieval before downstream LLM reasoning. Both represent important recent directions in efficient KGQA. However, differences in graph construction, retrieval targets, reasoners, hardware, and timing boundaries prevent their published runtimes from being used as direct evidence for AFP’s efficiency.

MoKGR [3] is methodologically relevant because it investigates query-adaptive graph pruning and reasoning-length selection. Nevertheless, its task formulation, datasets, and evaluation protocol differ from the WebQSP/CWQ natural-language KGQA setting considered here. It is therefore used as conceptual evidence for adaptive graph exploration rather than as a direct numerical baseline.

The contribution of AdaPruner-KGQA should consequently be interpreted as a lightweight retrieval-level extension of RoG. Its objective is not to replace the RoG planner or reasoner but to reduce residual entity/path branching after relation-level planning has already constrained the search.

6.9 Limitations and Threats to Validity

Several limitations should be considered when interpreting the results. First, AdaPruner-KGQA operates downstream of the RoG relation planner. Consequently, AFP cannot recover an answer path when the necessary relation sequence is absent from all generated plans. The pruning results therefore measure efficiency improvements within the RoG retrieval setting rather than across all possible KGQA search strategies.

Second, branch-level supervision is derived from training answers and the remaining relation-plan suffix. Although this creates task-specific pruning labels, incomplete answer annotation may cause some semantically reasonable alternative branches to be treated as negative.

Third, Search-Space Reduction is only one indicator of retrieval efficiency. AFP operates after the current candidate set has already been generated and therefore

cannot recover graph cost already incurred at that hop. Feature extraction, branch scoring, uncertainty estimation, and adaptive selection also introduce additional computation. Edges Examined is therefore interpreted together with Answer Retention rather than as an isolated measure of end-to-end efficiency.

Fourth, the unchanged RoG generation contract occasionally produces sequences whose total input and generated length exceeds the nominal 4,096-token context and some outputs reach the 512-token generation cap. Because the same reasoner contract is applied to both RoG and AFP, the comparison remains controlled, but these cases limit conclusions about generation robustness.

Finally, adaptive pruning introduces an unavoidable efficiency–recall trade-off. More aggressive pruning can increase SSR but can also remove answer-supporting intermediate prefixes. The results are therefore interpreted using Answer Retention, coverage, Hits@1, F1, and search reduction together rather than treating search reduction alone as evidence of improvement.

Chapter 7

Conclusion and Future Work

This final chapter sums up the main findings of AdaPruner-KGQA and outlines directions for extending adaptive search-space pruning in multi-hop KGQA.

7.1 Conclusion

This work investigated the residual search space that remains within RoG’s relation-constrained retrieval and introduced AdaPruner-KGQA, which integrates an Adaptive Frontier Pruner (AFP) into the retrieval stage while preserving the original relation planner and downstream reasoner.

The experiments show that relation-level planning does not eliminate entity- and path-level branching. Search distributions are strongly skewed, with substantially heavier extreme cases on CWQ, reaching up to 15,624 candidate branches and 43,473 examined edges. This confirms that meaningful search growth can remain even when the relation sequence is already fixed.

Under the frozen test configuration, AFP reduces examined-edge search cost by 2.24% on WebQSP and 5.05% on CWQ while retaining 99.20% and 98.72% of RoG-reachable answers, respectively. It also reduces retrieved reasoning paths by 21.8% on WebQSP and 27.0% on CWQ, with corresponding reductions of 10.6% and 6.2% in reasoner-input tokens.

Final KGQA performance remains close to the reproduced RoG baseline. On WebQSP, Hits@1 changes from 86.1% to 85.7% and F1 from 70.1% to 69.9%. On CWQ, Hits@1 changes from 60.9% to 60.4% and F1 from 53.7% to 53.2%. The ablation study further shows that uncertainty adaptation and temperature scaling have the strongest influence on AFP’s pruning behavior. Overall, the results

demonstrate that adaptive intermediate-frontier pruning can reduce residual graph exploration while largely preserving answer reachability and downstream QA effectiveness.

7.2 Future Work

Several directions can extend this work.

First, future research can develop stronger uncertainty-calibration methods so that pruning aggressiveness better reflects ambiguity in the candidate frontier.

Second, AFP can be evaluated across additional knowledge graphs, KGQA benchmarks, and retrieval architectures to determine whether the learned branch-selection behavior transfers beyond the RoG setting.

Third, branch supervision can be improved through weaker or self-supervised signals, reducing dependence on gold-answer-based suffix reachability during training.

Fourth, pruning strength can be conditioned more explicitly on question complexity, reasoning depth, frontier size, and local graph structure so that the retrieval budget varies with the expected difficulty of each reasoning step.

Finally, relation planning and frontier pruning could be jointly optimized so that relation-level and entity-level search control cooperate while preserving the modularity and interpretability of the current framework.

References

- [1] Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan, *Reasoning on Graphs: Faithful and Interpretable Large Language Model Reasoning*, In: Proceedings of the International Conference on Learning Representations (ICLR 2024), 2024.
- [2] Mufei Li, Siqi Miao, and Pan Li, *Simple Is Effective: The Roles of Graphs and Large Language Models in Knowledge-Graph-Based Retrieval-Augmented Generation*, In: Proceedings of the International Conference on Learning Representations (ICLR 2025), 2025.
- [3] Enjun Du, Siyi Liu, and Yongqi Zhang, *Mixture of Length and Pruning Experts for Knowledge Graphs Reasoning*, In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP 2025), pp. 432–453, 2025.
- [4] Apoorv Saxena, Aditya Tripathi, and Partha Talukdar, *Improving Multi-hop Question Answering over Knowledge Graphs Using Knowledge Base Embeddings*, In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020), pp. 4498–4507, 2020, <https://aclanthology.org/2020.acl-main.412.pdf>.
- [5] Haitian Sun, Bhuwan Dhingra, Manzil Zaheer, Kathryn Mazaitis, Ruslan Salakhutdinov, and William Cohen, *Open Domain Question Answering Using Early Fusion of Knowledge Bases and Text*, In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP 2018), pp. 4231–4242, 2018, <https://aclanthology.org/D18-1455.pdf>.
- [6] Haitian Sun, Tania Bedrax-Weiss, and William Cohen, *PullNet: Open Domain Question Answering with Iterative Retrieval on Knowledge Bases*

- and Text*, In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP 2019), pp. 2380–2390, 2019, <https://aclanthology.org/D19-1242.pdf>.
- [7] Weiqiang Jin, Biao Zhao, Hang Yu, Xi Tao, Ruiping Yin, and Guizhong Liu, *Improving Embedded Knowledge Graph Multi-hop Question Answering by Introducing Relational Chain Reasoning*, Data Mining and Knowledge Discovery 37(1), 255–288, 2022, doi:<https://doi.org/10.1007/s10618-022-00891-8>.
 - [8] Wen-Tau Yih, Matthew Richardson, Christopher Meek, Ming-Wei Chang, and Jina Suh, *The Value of Semantic Parse Labeling for Knowledge Base Question Answering*, In: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL 2016), pp. 201–206, 2016, <https://aclanthology.org/P16-2033.pdf>.
 - [9] Microsoft, *WebQuestions Semantic Parses Dataset*, Microsoft Download Center, Version 1.0, <https://www.microsoft.com/en-us/download/details.aspx?id=52763>, accessed August 29, 2026.
 - [10] Yuan Cu, *Complex Web Questions Dataset*, GitHub repository, <https://github.com/yuancu/complex-web-questions-dataset/>, accessed August 29, 2026.
 - [11] Kurt Bollacker, Colin Evans, Praveen Paritosh, Tim Sturge, and Jamie Taylor, *Freebase*, In: Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data (SIGMOD 2008), 2008, doi:<https://doi.org/10.1145/1376616.1376746>.
 - [12] Costas Mavromatis and George Karypis, *GNN-RAG: Graph Neural Retrieval for Efficient Large Language Model Reasoning on Knowledge Graphs*, In: Findings of the Association for Computational Linguistics: ACL 2025, pp. 16682–16699, 2025.
 - [13] Hugo Touvron et al., *Llama 2: Open Foundation and Fine-Tuned Chat Models*, arXiv preprint arXiv:2307.09288, 2023.
 - [14] Alexander Miller, Adam Fisch, Jesse Dodge, Amir-Hossein Karimi, Antoine Bordes, and Jason Weston, *Key-Value Memory Networks for Directly Reading Documents*, In: Proceedings of the 2016 Conference on Empirical Meth-

- ods in Natural Language Processing (EMNLP 2016), pp. 1400–1409, 2016, <https://aclanthology.org/D16-1147.pdf>.
- [15] Gaole He, Yunshi Lan, Jing Jiang, Wayne Xin Zhao, and Ji-Rong Wen, *Improving Multi-hop Knowledge Base Question Answering by Learning Intermediate Supervision Signals*, 2021, doi:<https://doi.org/10.1145/3437963.3441753>.
 - [16] OpenAI, *ChatGPT*, 2023. Available at: <https://openai.com/chatgpt>.
 - [17] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa, *Large Language Models are Zero-Shot Reasoners*, Advances in Neural Information Processing Systems (NeurIPS), vol. 35, 2022.
 - [18] Apoorv Saxena, Adrian Kochsiek, and Rainer Gemulla, *Sequence-to-Sequence Knowledge Graph Completion and Question Answering*, In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL 2022), Volume 1: Long Papers, pp. 2814–2828, 2022, doi:<https://doi.org/10.18653/v1/2022.acl-long.201>.
 - [19] Jonathan Berant, Andrew Chou, Roy Frostig, and Percy Liang, *Semantic Parsing on Freebase from Question-Answer Pairs*, In: Proceedings of EMNLP 2013, pp. 1533–1544, 2013, <https://aclanthology.org/D13-1160.pdf>.
 - [20] Shuang Chen, Qian Liu, Zhiwei Yu, Chin-Yew Lin, Jian-Guang Lou, and Feng Jiang, *ReTraCk: A Flexible and Efficient Framework for Knowledge Base Question Answering*, In: Proceedings of ACL-IJCNLP 2021 (System Demonstrations), 2021, doi:<https://doi.org/10.18653/v1/2021.acl-demo.39>.
 - [21] Yu Gu and Yu Su, *ArcaneQA: Dynamic Program Induction and Contextualized Encoding for Knowledge Base Question Answering*, arXiv preprint arXiv:2204.08109, 2022.
 - [22] Donghan Yu, Sheng Zhang, Patrick Ng, Henghui Zhu, Alexander Hanbo Li, Jun Wang, Yiqun Hu, William Wang, Zhiguo Wang, and Bing Xiang, *DECAF: Joint Decoding of Answers and Logical Forms for Question Answering over Knowledge Bases*, In: Proceedings of the International Conference on Learning Representations (ICLR 2023), 2023.

- [23] Hyung Won Chung, Le Hou, Shayne Longpre, et al., *Scaling Instruction-Finetuned Language Models*, Journal of Machine Learning Research, vol. 25, pp. 1–53, 2024.
- [24] Keheng Wang, Feiyu Duan, Sirui Wang, Peiguang Li, Yunsen Xian, Chuantao Yin, Wenge Rong, and Zhang Xiong, *Knowledge-Driven CoT: Exploring Faithful Reasoning in LLMs for Knowledge-Intensive Question Answering*, arXiv preprint arXiv:2308.13259, 2023.
- [25] Jinhao Jiang, Kun Zhou, Wayne Xin Zhao, and Ji-Rong Wen, *UniKGQA: Unified Retrieval and Reasoning for Solving Multi-hop Question Answering over Knowledge Graph*, arXiv preprint arXiv:2212.00959, 2022.
- [26] Yunshi Lan and Jing Jiang, *Query Graph Generation for Answering Multi-hop Complex Questions from Knowledge Bases*, In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL), pp. 969–974, 2020, doi:<https://doi.org/10.18653/v1/2020.acl-main.91>.
- [27] Yawei Sun, Lingling Zhang, Gong Cheng, and Yuzhong Qu, *SPARQA: Skeleton-Based Semantic Parsing for Complex Questions over Knowledge Bases*, In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 34, no. 05, pp. 8952–8959, 2020, doi:<https://doi.org/10.1609/aaai.v34i05.6426>.
- [28] Jiho Kim, Yeonsu Kwon, Yohan Jo, and Edward Choi, *KG-GPT: A General Framework for Reasoning on Knowledge Graphs Using Large Language Models*, In: Findings of the Association for Computational Linguistics: EMNLP 2023, pp. 9410–9421, 2023, <https://aclanthology.org/2023.findings-emnlp.631.pdf>.
- [29] Yunshi Lan, Gaole He, Jinhao Jiang, Jing Jiang, Wayne Xin Zhao, and Ji-Rong Wen, *Complex Knowledge Base Question Answering: A Survey*, arXiv preprint arXiv:2108.06688, 2021.
- [30] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto, *Stanford Alpaca: An Instruction-Following LLaMA Model*, GitHub repository, 2023.

- [31] Derong Xu, Xinhang Li, Ziheng Zhang, Zhenxi Lin, Zhihong Zhu, Zhi Zheng, Xian Wu, Xiangyu Zhao, Tong Xu, and Enhong Chen, *Harnessing Large Language Models for Knowledge Graph Question Answering via Adaptive Multi-Aspect Retrieval-Augmentation*, Proceedings of the AAAI Conference on Artificial Intelligence, vol. 39, no. 24, pp. 25570–25578, 2025.
- [32] Ying Su, Jipeng Zhang, Yangqiu Song, and Tong Zhang, *PipeNet: Question Answering with Semantic Pruning over Knowledge Graphs*, Proceedings of the 13th Joint Conference on Lexical and Computational Semantics (*SEM), pp. 360–371, Association for Computational Linguistics, 2024.
- [33] Costas Mavromatis and George Karypis, *ReaRev: Adaptive Reasoning for Question Answering over Knowledge Graphs*, In: Findings of the Association for Computational Linguistics: EMNLP 2022, pp. 2447–2460, 2022, <https://aclanthology.org/2022.findings-emnlp.181.pdf>.
- [34] Jiaxin Shi, Shulin Cao, Lei Hou, Juanzi Li, and Hanwang Zhang, *TransferNet: An Effective and Transparent Framework for Multi-hop Question Answering over Relation Graph*, In: Proceedings of EMNLP 2021, pp. 4149–4158, 2021, <https://aclanthology.org/2021.emnlp-main.341.pdf>.
- [35] Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Heung-Yeung Shum, and Jian Guo, *Think-on-Graph: Deep and Responsible Reasoning of Large Language Model with Knowledge Graph*, arXiv preprint arXiv:2307.07697, 2023.
- [36] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 5776–5788, 2020. [Online]. Available: <https://arxiv.org/abs/2002.10957>
- [37] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China, pp. 3982–3992, 2019. [Online]. Available: <https://aclanthology.org/D19-1410/>

- [38] Xi Ye, Semih Yavuz, Kazuma Hashimoto, Yingbo Zhou, and Caiming Xiong, *RnG-KBQA: Generation Augmented Iterative Ranking for Knowledge Base Question Answering*, arXiv preprint arXiv:2109.08678, 2022.
- [39] Jing Zhang, Xiaokang Zhang, Jifan Yu, Jian Tang, Jie Tang, Cuiping Li, and Hong Chen, *Subgraph Retrieval Enhanced Model for Multi-hop Knowledge Base Question Answering*, In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2022, doi:<https://aclanthology.org/2022.acl-long.396v2.pdf>.
- [40] Ruilin Zhao, Feng Zhao, Long Wang, Xianzhi Wang, and Guandong Xu, *KG-CoT: Chain-of-Thought Prompting of Large Language Models over Knowledge Graphs for Knowledge-Aware Question Answering*, In: Proceedings of IJCAI 2024, 2024, <https://www.ijcai.org/proceedings/2024/0734.pdf>.
- [41] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann, *Knowledge Graphs*, ACM Computing Surveys 54(4), Article 71, 2021, doi:<https://doi.org/10.1145/3447772>.
- [42] Liyi Chen, Panrong Tong, Zhongming Jin, Ying Sun, Jieping Ye, and Hui Xiong, *Plan-on-Graph: Self-Correcting Adaptive Planning of Large Language Model on Knowledge Graphs*, In: Advances in Neural Information Processing Systems (NeurIPS 2024), 2024, <https://arxiv.org/abs/2410.23875>.

Appendix A

A1 Official Implementation Links of Compared Methods

Model	Official Code Link
Semantic Parsing-Based KGQA	
Berant et al. [19]	http://nlp.stanford.edu/software/sempre
Yih et al. [8]	http://aka.ms/WebQSP
ArcaneQA [21]	https://github.com/dki-lab/ArcaneQA
RnG-KBQA [38]	https://github.com/salesforce/rng-kbqa
Information Retrieval-Based Multi-Hop KGQA	
GraftNet [5]	https://github.com/haitian-sun/GraftNet
PullNet [6]	https://github.com/haitian-sun/GraftNet
UniKGQA [25]	https://github.com/RUCAIBox/UniKGQA
SR+NSM [39]	https://github.com/RUCKBReasoning/SubgraphRetrievalKBQA
PipeNet [32]	https://github.com/HKUST-KnowComp/PipeNet
SubgraphRAG [2]	https://github.com/awslabs/SubgraphRAG
GNN-RAG [12]	https://github.com/cmavro/GNN-RAG
Embedding-Based and Hybrid KGQA	
KV-Mem [14]	https://github.com/facebookarchive/MemNN/tree/master/KvMemnn
EmbedKGQA [4]	https://github.com/malllabiisc/EmbedKGQA
NSM [15]	https://github.com/RichardHGL/WSDM2021_NSM
TransferNet [34]	https://github.com/shijx12/TransferNet
KGT5 [18]	https://github.com/cambridgelt1/KGT5
Rce-KGQA [7]	https://github.com/albert-jin/Rce-KGQA
ReaRev [33]	https://github.com/cmavro/ReaRev_KGQA
LLM-Based Multi-Hop KGQA	
LLaMA2-Chat-7B [13]	https://github.com/meta-llama/llama
ChatGPT [16]	https://openai.com/chatgpt
ChatGPT+CoT [17]	https://github.com/kojima-takeshi188/zero_shot_cot
LLM with KG-Based Multi-Hop KGQA	
KG-GPT [28]	https://github.com/jiho283/KG-GPT
DECAF [22]	https://github.com/awslabs/decode-answer-logical-form
Think-on-Graph [35]	https://github.com/GasolSun36/ToG
Plan-on-Graph [42]	https://github.com/liyichen-cly/PoG
RoG [1]	https://github.com/zjunlp/rog
AMAR [31]	https://github.com/Applied-Machine-Learning-Lab/AMAR

Table 1: Official implementation links of compared methods